SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

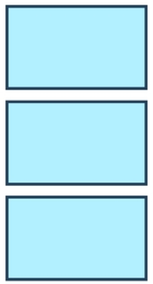
분할 정복

Python

학습시작

퍼즐을 맞추는 가장 효율적인 순서는?

퍼즐 100조각을 한 번에 맞추기 어렵습니다. 그래서 아래와 같은 단계를 고려해보았습니다. 아래 전략은 어떤 순서로 이루어졌을까요?



: 색깔이나 테두리 등으로 조각을 분류함

: 각 분류된 조각 그룹을 따로따로 맞춤

: 맞춘 조각 그룹을 하나로 이어 전체 퍼즐 완성

- 분할정복을 이해하면, 정렬이나 탐색 문제를 더 효율적으로 풀 수 있어요.
- 복잡한 문제를 나누고 → 각 부분을 해결하고 → 다시 합치는 구조
- 이 전략은 병합정렬, 퀵정렬, 이진탐색 등에서 사용되는 알고리즘 개념과 연결돼요!

Objectives

학습목표

- 문제를 분할해서 해결하는 분할 정복 기법을 이해한다.
- 분할 정복 기법을 적용된 대표적인 정렬 알고리즘인 퀵 정렬과 병합 정렬에 대해 학습한다.
- 정렬된 자료들을 분할해서 탐색하는 이진 검색에 대해 학습한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

분할 정복

- 분할 정복 기법
- 병합 정렬
- 퀵 정렬
- 이진 검색

분할 정복

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

분할 정복 기법

분할 정복 기법의 개념

분할 정복

문제를 작은 하위 문제로 나누고(분할) 각각을 해결(정복)한 뒤,
그 결과를 결합(통합)하여 원래 문제를 해결하는 알고리즘 기법

분할 정복 기법의 유래

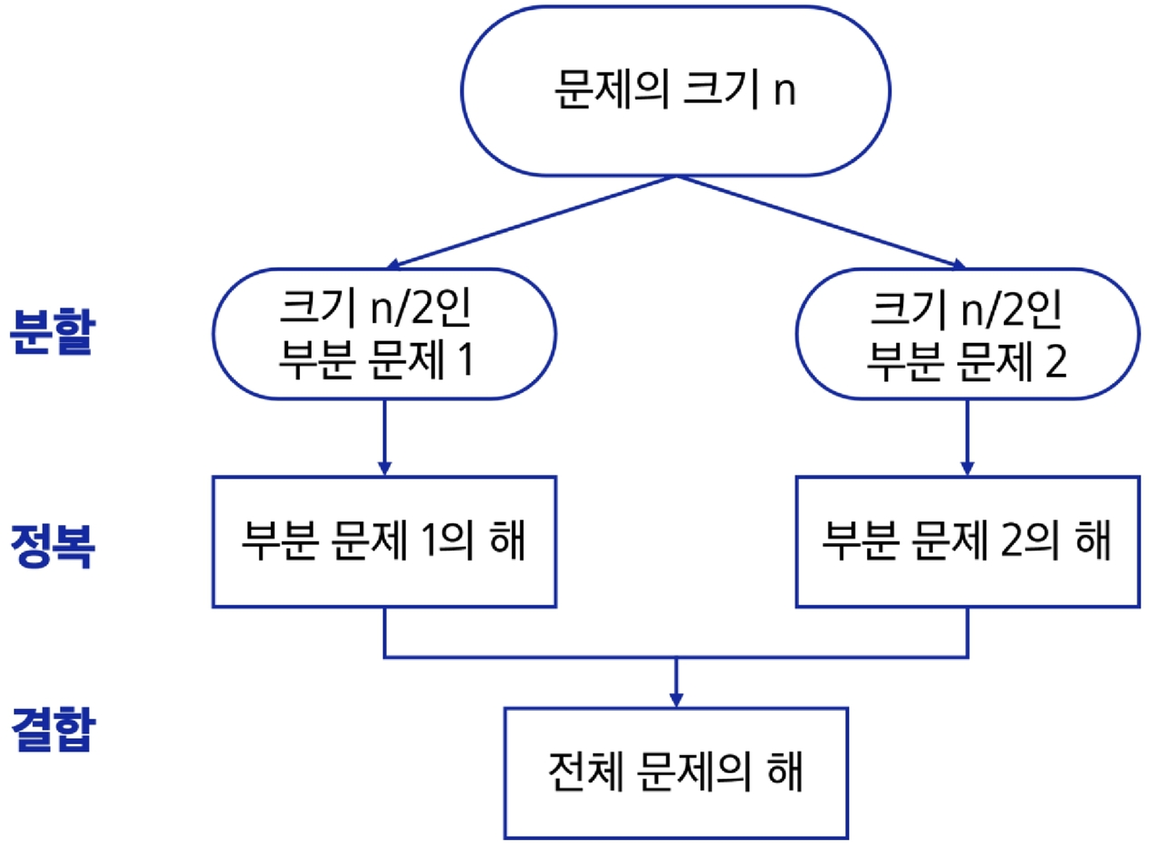
- 1805년 12월 2일 아우스터리츠 전투에서 나폴레옹이 사용한 전략임
- 전력이 우세한 연합군을 공격하기 위해 나폴레옹은 연합군의 중앙부로 쳐들어가 연합군을 둘로 나눔
- 둘로 나뉜 연합군을 한 부분씩 격파함

분할 정복 기법의 설계 전략

- 분할(Divide) : 해결할 문제를 여러 개의 작은 부분으로 나눔
- 정복(Conquer) : 나눈 작은 문제를 각각 해결
- 통합(Combine) : (필요하다면) 해결된 해답을 모음

분할 정복 기법의 구조

- Top-down approach 예시



<그림1_알고리즘 응용_분할정복_분할 정복 기법의 구조>

분할 정복 기법의 예시

- 가짜 동전 찾기

n 개의 동전들 중에 가짜 동전이 하나 포함되어 있다. 가짜 동전은 진짜 동전에 비해 아주 조금 가볍다.
진짜 동전들의 무게가 동일하다고 할 때 양팔 저울을 이용해서 가짜 동전을 찾아보자.

- 양팔 저울을 최소로 사용해서 가짜 동전을 찾는 방법은 무엇인가?
- 예를 들어 동전이 24(진짜 23, 가짜 1)개 있다면?

분할 정복 기법의 예시 - 거듭 제곱

분할 정복 기법을 이해하기 위해, 자연수 C의 n 제곱 값을 구하는 함수를 구현해봅시다

분할 정복 기법의 예시 - 거듭 제곱

- 반복(Iterative) 알고리즘 : O(n)

$$C^2 = C \times C$$
$$C^3 = C \times C \times C$$
$$\dots$$
$$C^n = C \times C \times C \times \cdots \times C$$

```
Iterative_Power(x, n)
    result ← 1

    FOR i in 1 → n
        result ← result * x

    RETURN result
```

분할 정복 기법의 예시 - 거듭 제곱

- 분할 정복 기반의 알고리즘 : O($\log_2 n$)

$$C^n = \begin{cases} C^{n/2} \bullet C^{n/2} & n\text{은짝수} \\ C^{(n-1)/2} \bullet C^{(n-1)/2} \bullet C & n\text{은홀수} \end{cases}$$

$$C^8 = C \times C \times C \times C \times C \times C \times C \times C$$
$$C^8 = C^4 \times C^4 = (C^4)^2 = \left((C^2)^2\right)^2$$

...

$$C^n = C^{\frac{n-1}{2}} \times C^{\frac{n-1}{2}} = \left(C^{\frac{n-1}{2}}\right)^2 \times C$$

```
Recursive_Power(x, n)
    IF n == 1 : RETURN x
    IF n is even
        y ← Recursive_Power(x, n/2)
        RETURN y * y
    ELSE
        y ← Recursive_Power(x, (n-1)/2)
        RETURN y * y * x
```

병합 정렬

병합 정렬(Merge Sort)의 개념

병합 정렬
Merge Sort

여러 개의 정렬된 자료의 집합을 병합하여 한 개의 정렬된 집합으로 만드는 방식

병합 정렬 과정

- 자료를 최소 단위의 문제까지 나눈 후에 차례대로 정렬하여 최종 결과를 얻어냄
- top-down 방식

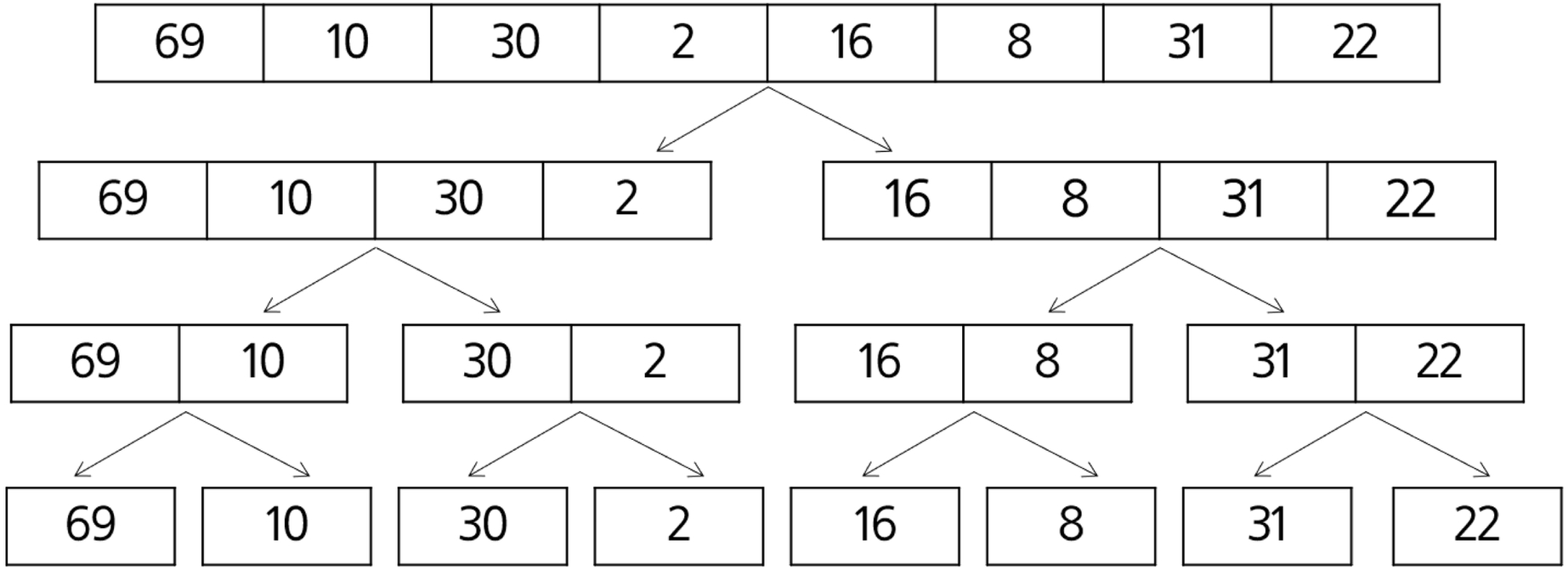
시간 복잡도

- O(n log n)

병합 정렬 과정 예시

- {69, 10, 30, 2, 16, 8, 31, 22}를 병합 정렬하는 과정

※ 분할 단계 : 전체 자료 집합에 대하여, 최소 크기의 부분집합이 될 때까지 분할 작업을 계속한다.

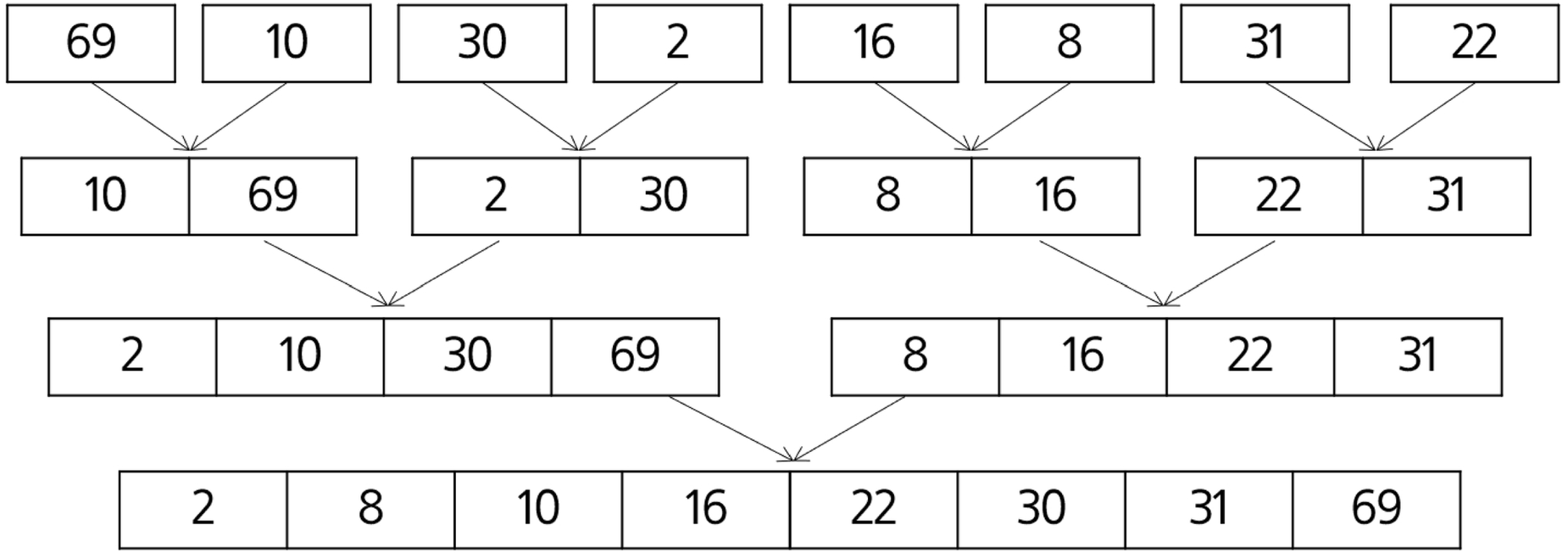


<그림2_알고리즘 응용_분할정복_병합 정렬 과정 1>

병합 정렬 과정 예시

- {69, 10, 30, 2, 16, 8, 31, 22}를 병합 정렬하는 과정

※ 병합 단계 : 2개의 부분집합을 정렬하면서 하나의 집합으로 병합한다. 8개의 부분집합이 1개로 병합될 때까지 반복



<그림3_알고리즘 응용_분할정복_병합 정렬 과정 2>

병합 정렬 알고리즘

- 분할 과정

- 코드 : 1.merge_sort.py

```
merge_sort(LIST m)
    IF length(m) == 1 : RETURN m

    LIST left, right
    middle ← length(m) / 2
    FOR x in m before middle
         add x to left
    FOR x in m after or equal middle
         add x to right

    left ← merge_sort(left)
    right ← merge_sort(right)

    RETURN merge(left, right)
```

병합 정렬 알고리즘

- 병합 과정

```
merge(LIST left, LIST right)
    LIST result

    WHILE length(left) > 0 OR length(right) > 0
        IF length(left) > 0 AND length(right) > 0
            IF first(left) <= first(right)
                append popfirst(left) to result
            ELSE
                append popfirst(right) to result
        ELIF length(left) > 0
            append posfirst(left) to result
        ELIF length(right) > 0
            append popfirst(right) to result
    RETURN result
```

퀵 정렬

퀵 정렬(Quick Sort)의 개념

퀵 정렬
Quick Sort

기준값을 중심으로 주어진 배열을 두 개로 분할하고,
각각을 정렬하여 전체 배열을 정렬하는 방식

병합 정렬과 퀵 정렬의 차이

	병합 정렬	퀵 정렬
분할 기준	단순히 배열을 반으로 나눔	기준 아이템(pivot item)을 중심으로 기준보다 작은 것을 왼편, 큰 것을 오른편에 위치 시킴
병합 처리	정렬된 부분을 다시 병합하는 과정이 필요함	별도의 병합 과정 불필요

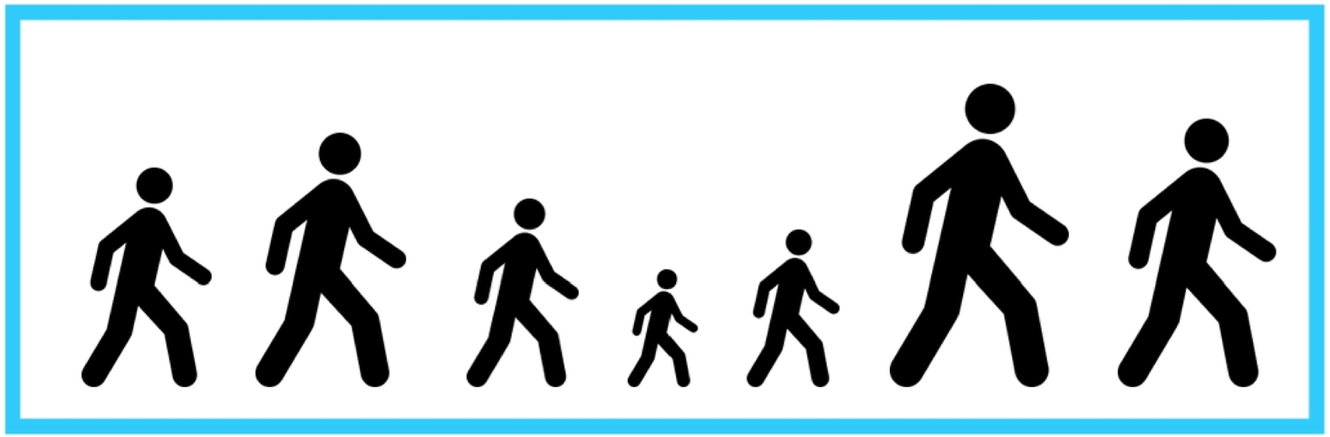
〈표1_알고리즘 응용_분할정복_병합정렬과 퀵정렬의 차이〉

시간 복잡도

- 평균 시간복잡도 O(nlogn)
- Partitioning 이라는 과정을 반복하면서, 빠른 속도로 정렬이 되는 알고리즘 입니다.

Partitioning

1. **작업 영역을 정한다.**
2. 작업 영역 중 가장 왼쪽에 있는 수를 Pivot 이라고 하자. (Pivot을 “기준”으로 해석한다.)
3. Pivot을 기준으로
   - 왼쪽에는 Pivot 보다 작은 수를 배치한다. (정렬 안됨)
   - 오른쪽에는 Pivot 보다 큰 수를 배치한다. (정렬 안됨)



작업 영역

<그림4_알고리즘 응용_분할정복_파티셔닝 1>

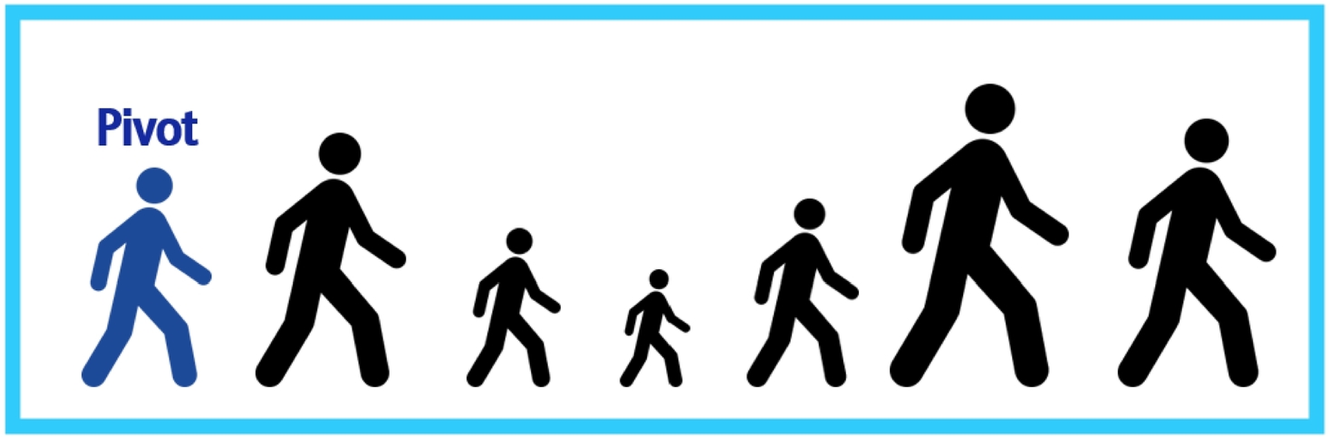
Partitioning

1. 작업 영역을 정한다.

2. 작업 영역 중 가장 왼쪽에 있는 수를 Pivot 이라고 하자. (Pivot을 “기준”으로 해석한다.)

3. Pivot을 기준으로
   - 왼쪽에는 Pivot 보다 작은 수를 배치한다. (정렬 안됨)
   - 오른쪽에는 Pivot 보다 큰 수를 배치한다. (정렬 안됨)

※ Pivot은 중간값, 우측 끝 값으로 설정해도 상관없지만, 우리는 왼쪽 끝 값을 기준으로 한다.

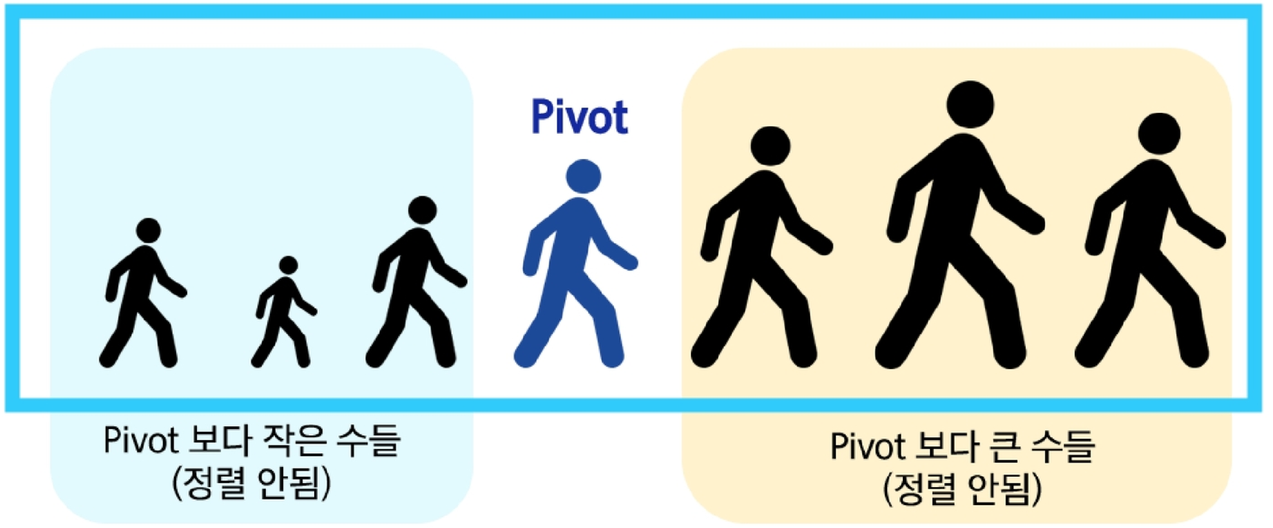


작업 영역

<그림5_알고리즘 응용_분할정복_파티셔닝 2>

Partitioning

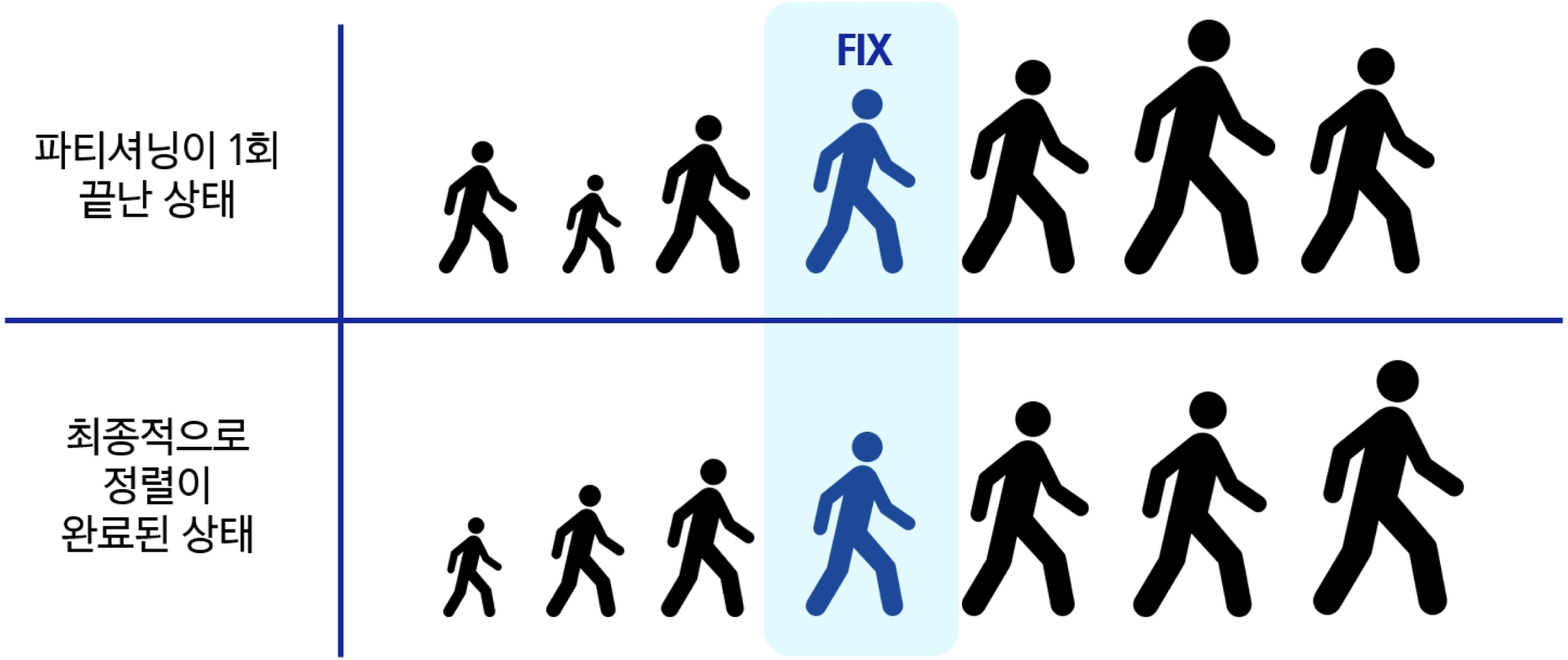
1. 작업 영역을 정한다.
2. 작업 영역 중 가장 왼쪽에 있는 수를 Pivot 이라고 하자. (Pivot을 “기준”으로 해석한다.)
3. **Pivot을 기준으로**
   - **왼쪽에는 Pivot 보다 작은 수를 배치한다. (정렬 안됨)**
   - **오른쪽에는 Pivot 보다 큰 수를 배치한다. (정렬 안됨)**



<그림6_알고리즘 응용_분할정복_파티셔닝 3>

Partitioning

- 파티셔닝이 끝나고 Pivot의 위치는 확정(Fix)됩니다.
- 즉, 정렬이 다 되었을 때에도 Pivot의 위치는 지금 위치 그대로 배정됩니다.

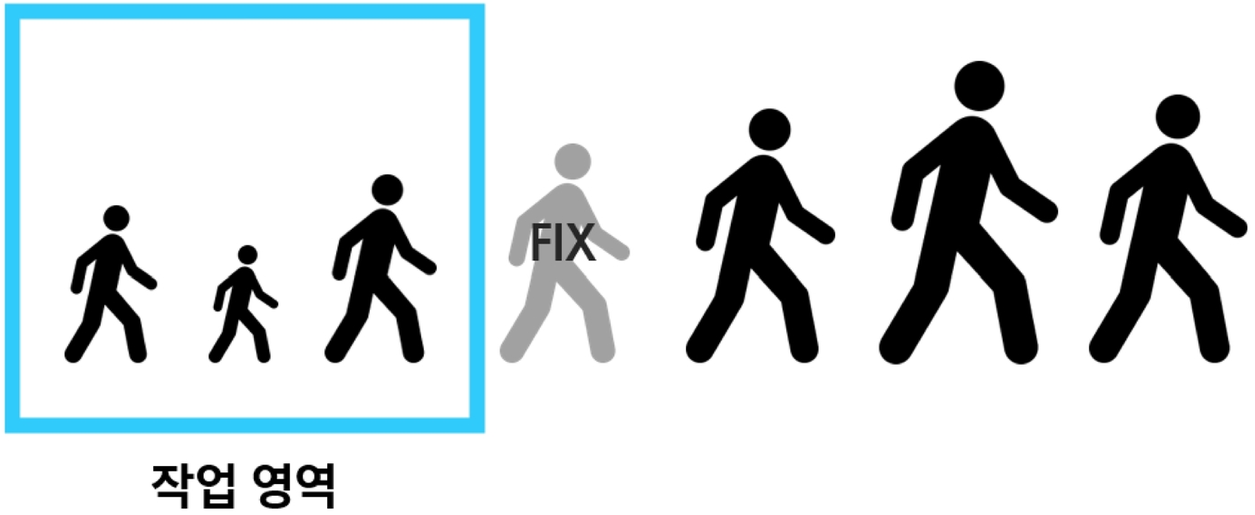


<그림7_알고리즘 응용_분할정복_파티셔닝 4>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. **작업 영역 지정**
2. Pivot 결정
3. Pivot을 기준으로 배치 변경

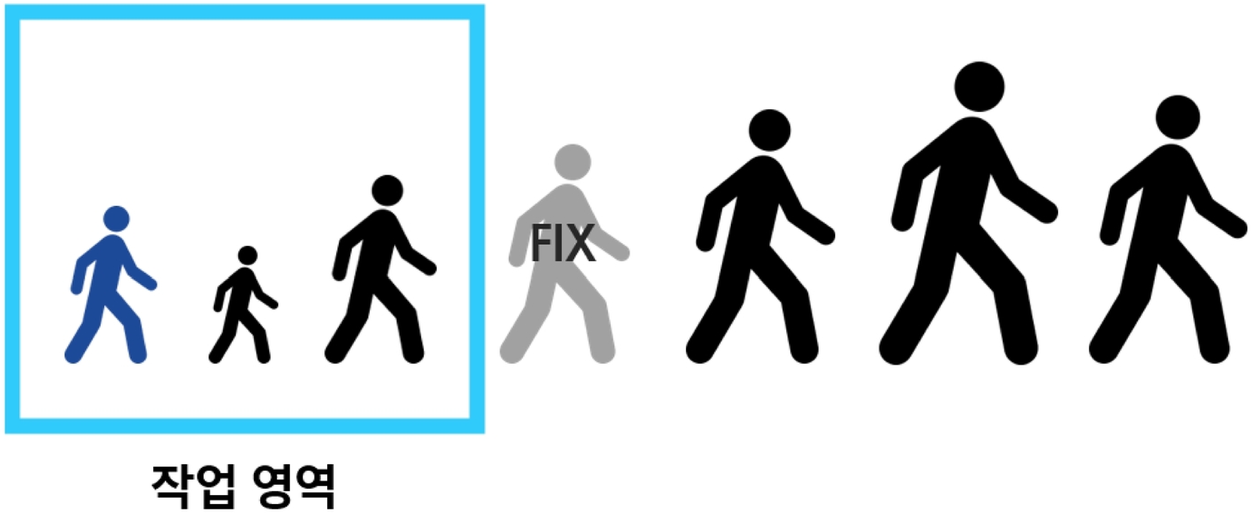


<그림8_알고리즘 응용_분할정복_파티셔닝 5>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. 작업 영역 지정
2. **Pivot 결정**
3. Pivot을 기준으로 배치 변경



<그림9_알고리즘 응용_분할정복_파티셔닝 6>

Partitioning

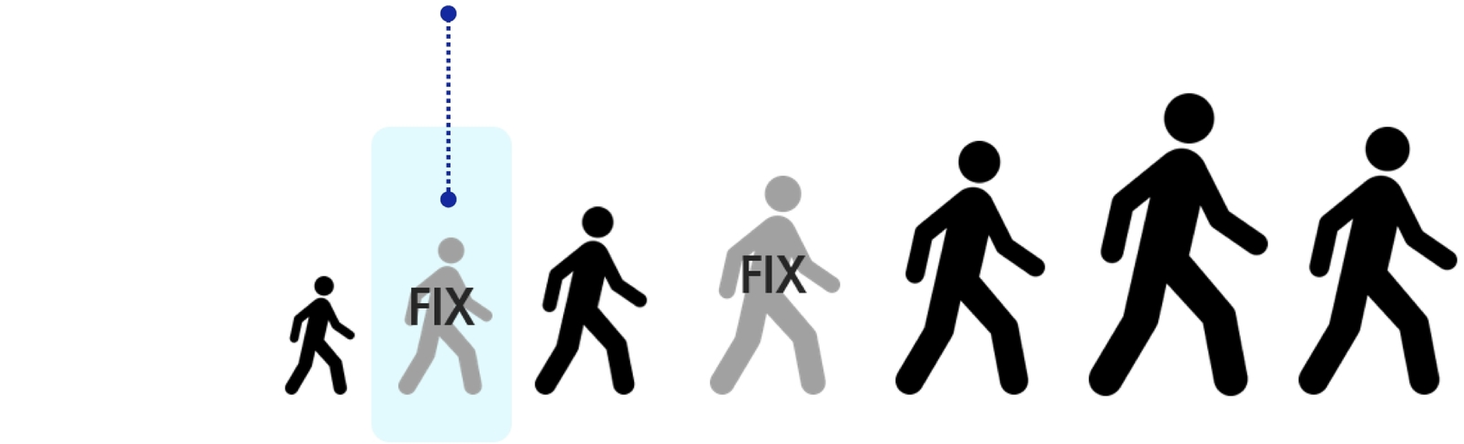
- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. 작업 영역 지정
2. Pivot 결정
3. **Pivot을 기준으로 배치 변경**

<그림10_알고리즘 응용_분할정복_파티셔닝 7>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.



<그림11_알고리즘 응용_분할정복_파티셔닝 8>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. **작업 영역 지정**
2. Pivot 결정
3. Pivot을 기준으로 배치 변경

<그림12_알고리즘 응용_분할정복_파티셔닝 9>

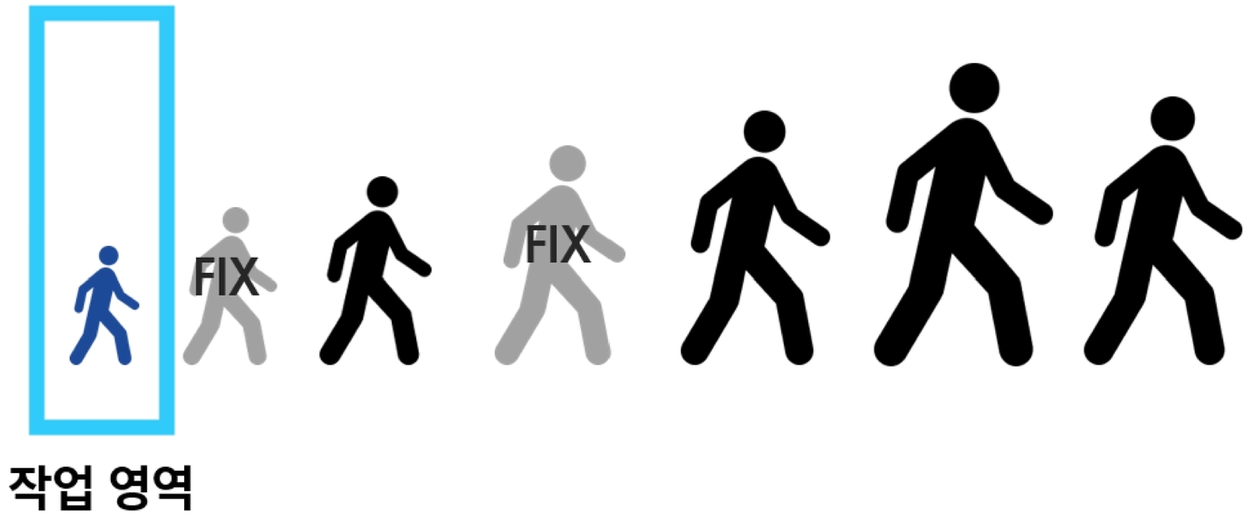
Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. 작업 영역 지정

2. Pivot 결정

3. Pivot을 기준으로 배치 변경



<그림13_알고리즘 응용_분할정복_파티셔닝 10>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. 작업 영역 지정

2. Pivot 결정

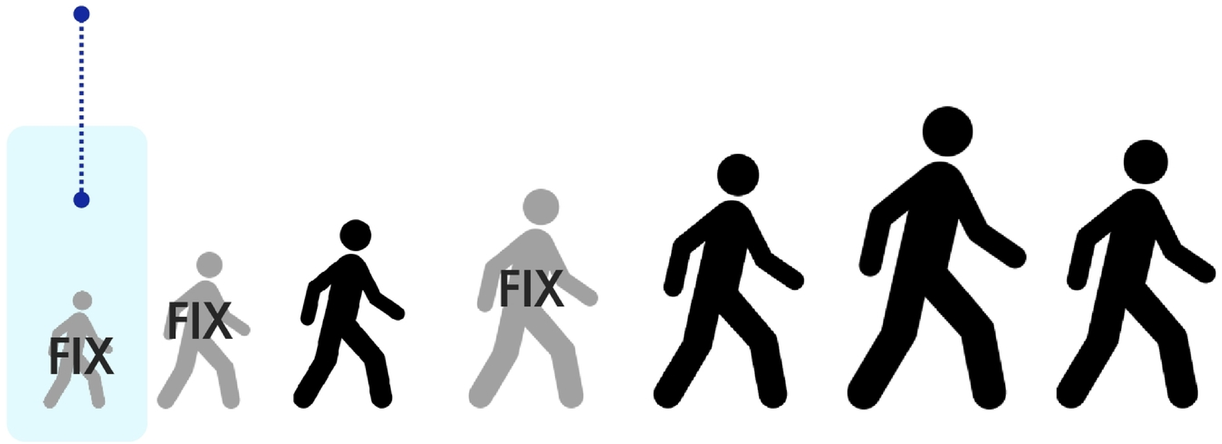
3. Pivot을 기준으로 배치 변경 ····● 작업 영역에 혼자이므로, 왼쪽과 오른쪽에 배치할 값이 없다.

<그림14_알고리즘 응용_분할정복_파티셔닝 11>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

파티셔닝이 끝났으니, Pivot의 위치는 확정(Fix)된다

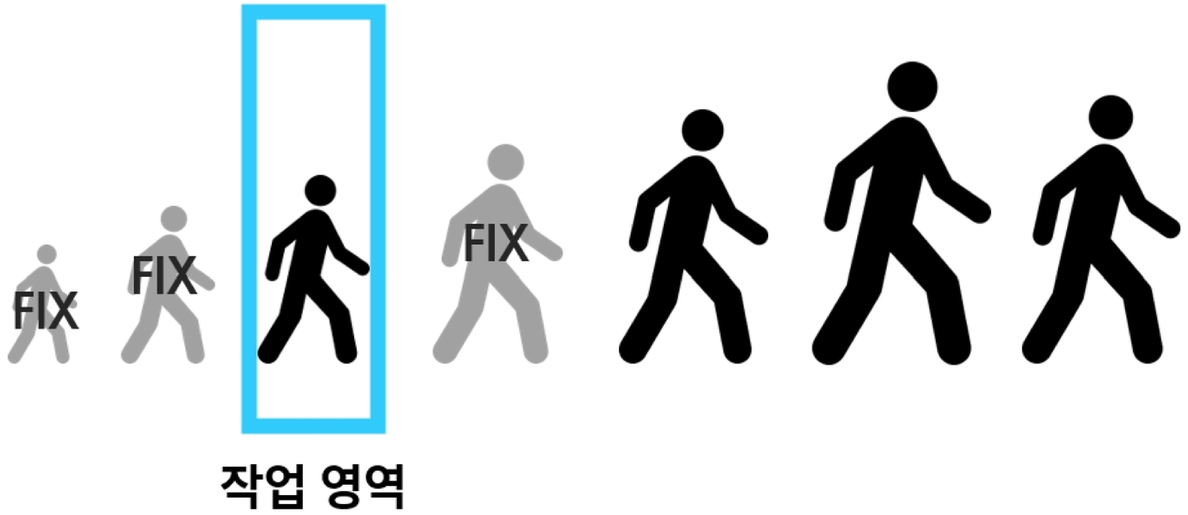


〈그림15_알고리즘 응용_분할정복_파티셔닝 12〉

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. **작업 영역 지정**
2. Pivot 결정
3. Pivot을 기준으로 배치 변경



<그림16_알고리즘 응용_분할정복_파티셔닝 13>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

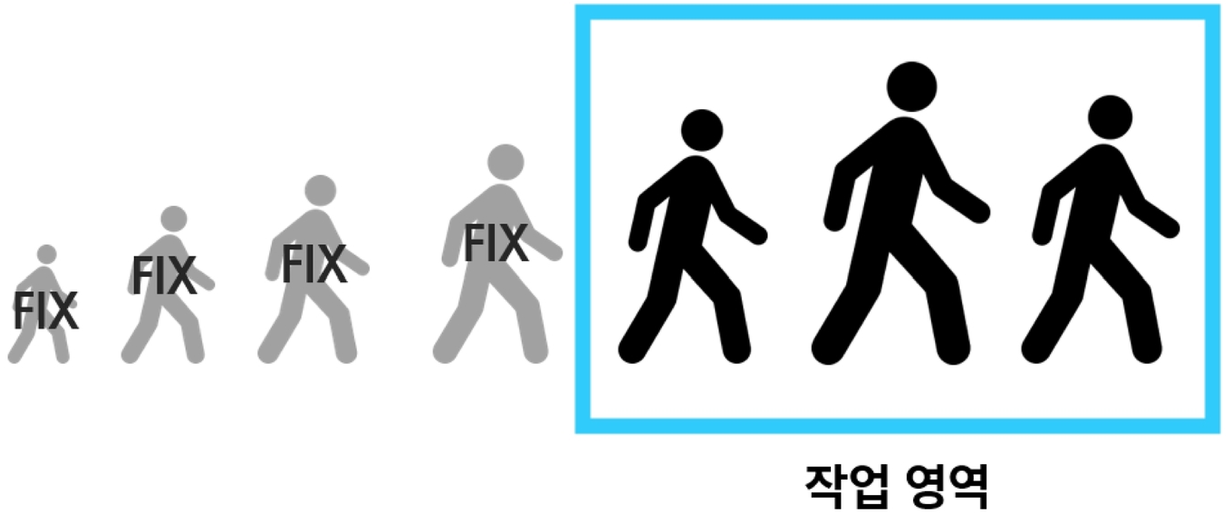
작업 영역에 혼자이므로, 해당 값이 Fix가 되고 바로 파티셔닝이 종료된다.

<그림17_알고리즘 응용_분할정복_파티셔닝 14>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. **작업 영역 지정**
2. Pivot 결정
3. Pivot을 기준으로 배치 변경



<그림18_알고리즘 응용_분할정복_파티셔닝 15>

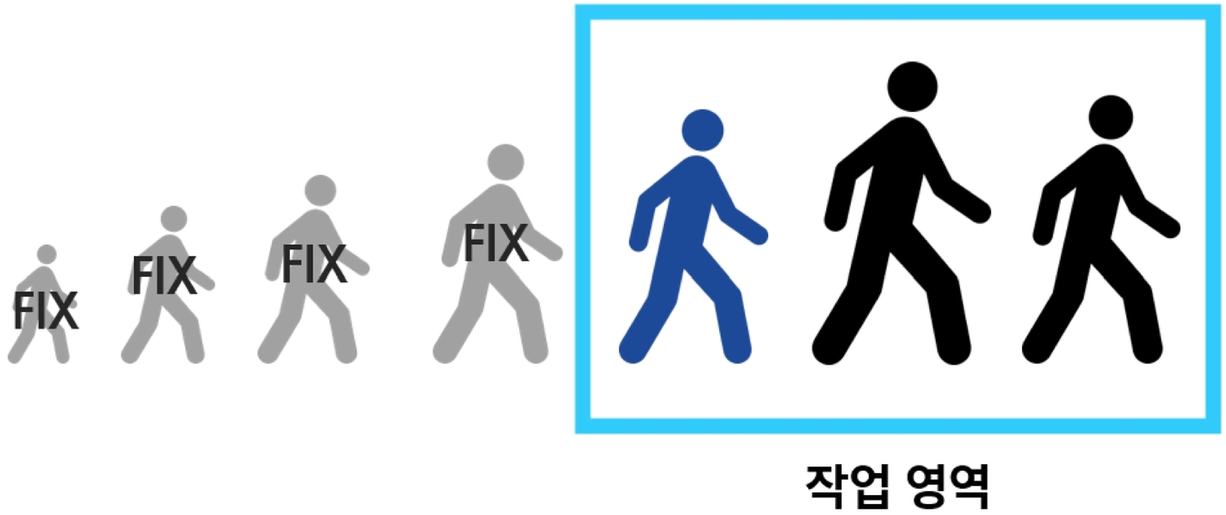
Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. 작업 영역 지정

2. Pivot 결정

3. Pivot을 기준으로 배치 변경



<그림19_알고리즘 응용_분할정복_파티셔닝 16>

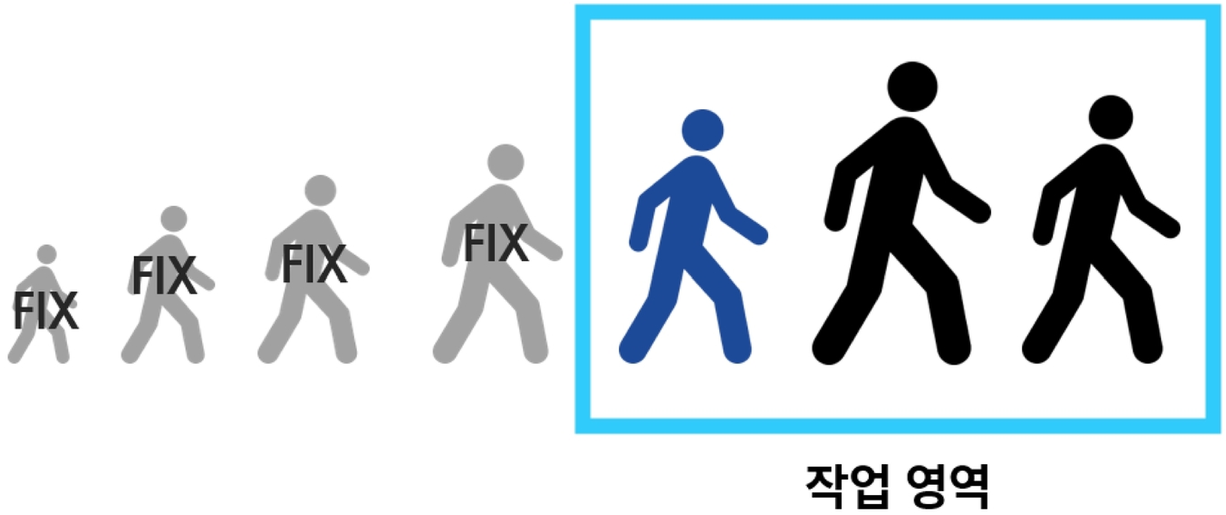
Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. 작업 영역 지정

2. Pivot 결정

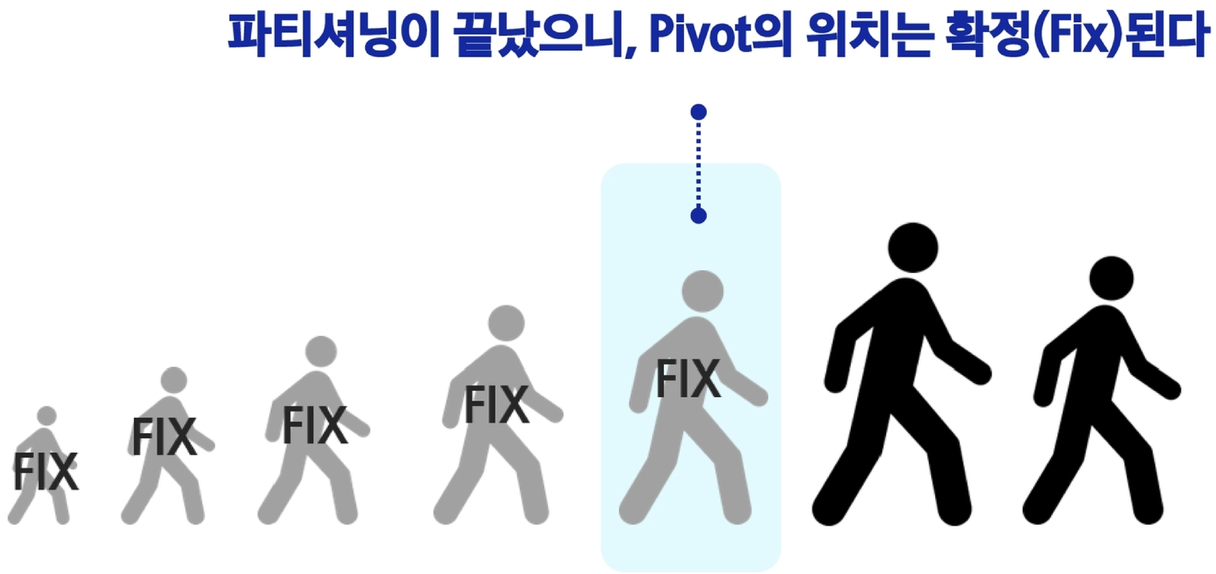
3. Pivot을 기준으로 배치 변경 ····● 우측에는 Pivot보다 모두 큰 값이므로 변화가 없다.



<그림20_알고리즘 응용_분할정복_파티셔닝 17>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

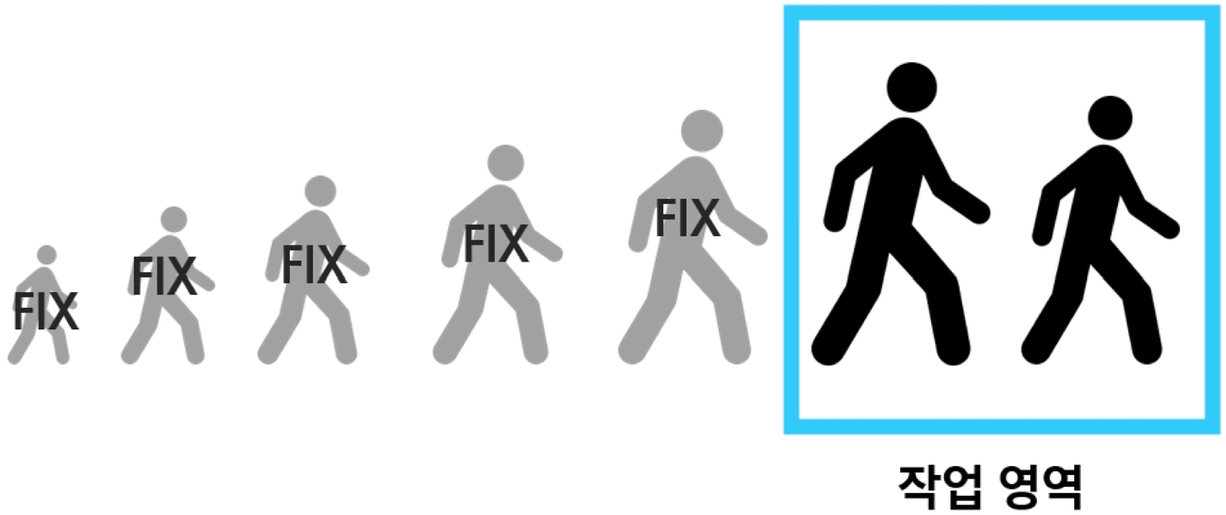


<그림21_알고리즘 응용_분할정복_파티셔닝 18>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. **작업 영역 지정**
2. Pivot 결정
3. Pivot을 기준으로 배치 변경



<그림22_알고리즘 응용_분할정복_파티셔닝 19>

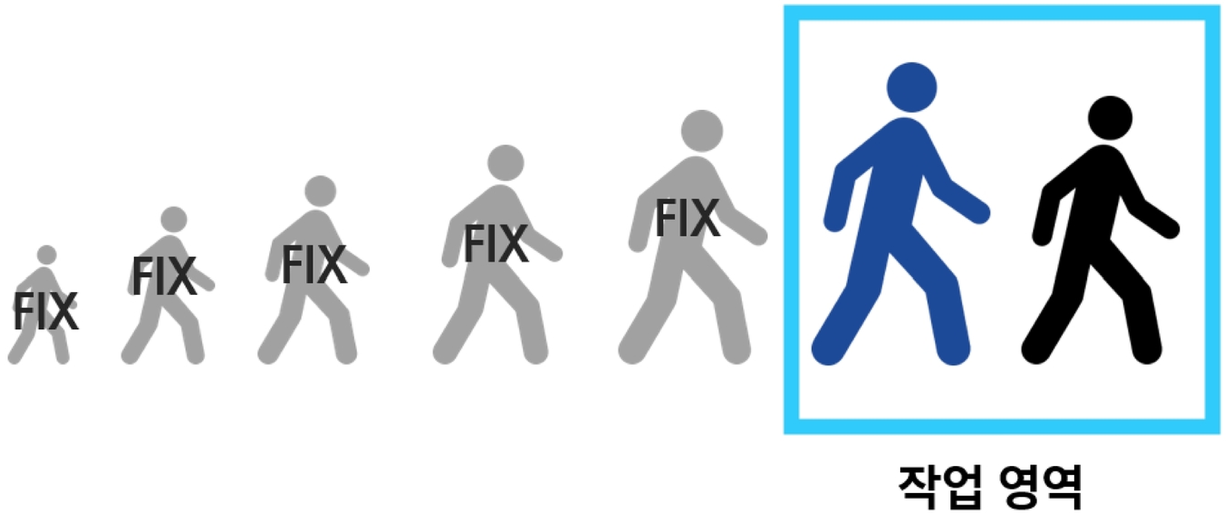
Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. 작업 영역 지정

2. Pivot 결정

3. Pivot을 기준으로 배치 변경

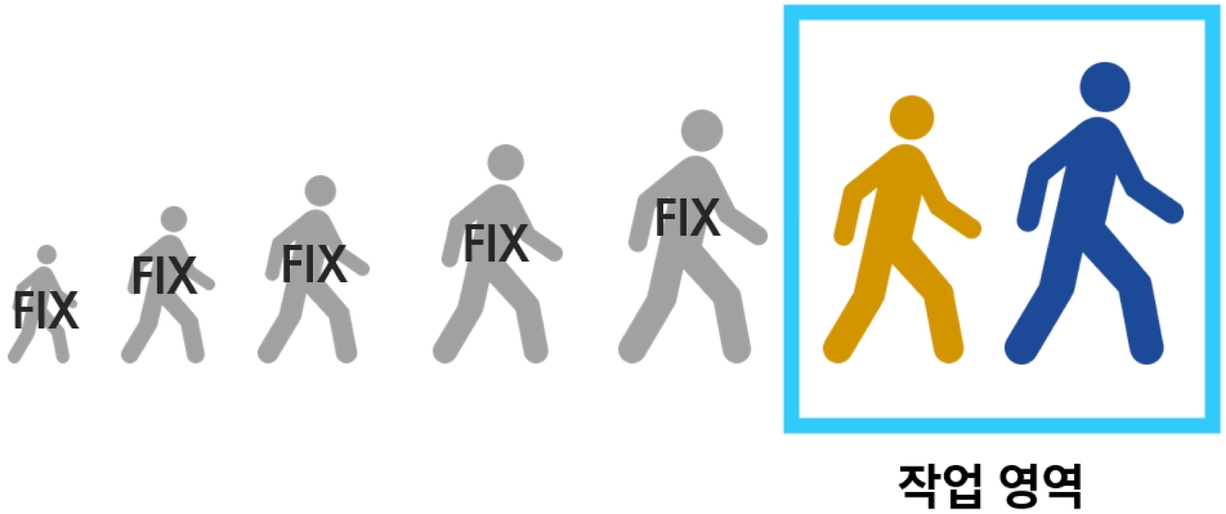


<그림23_알고리즘 응용_분할정복_파티셔닝 20>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

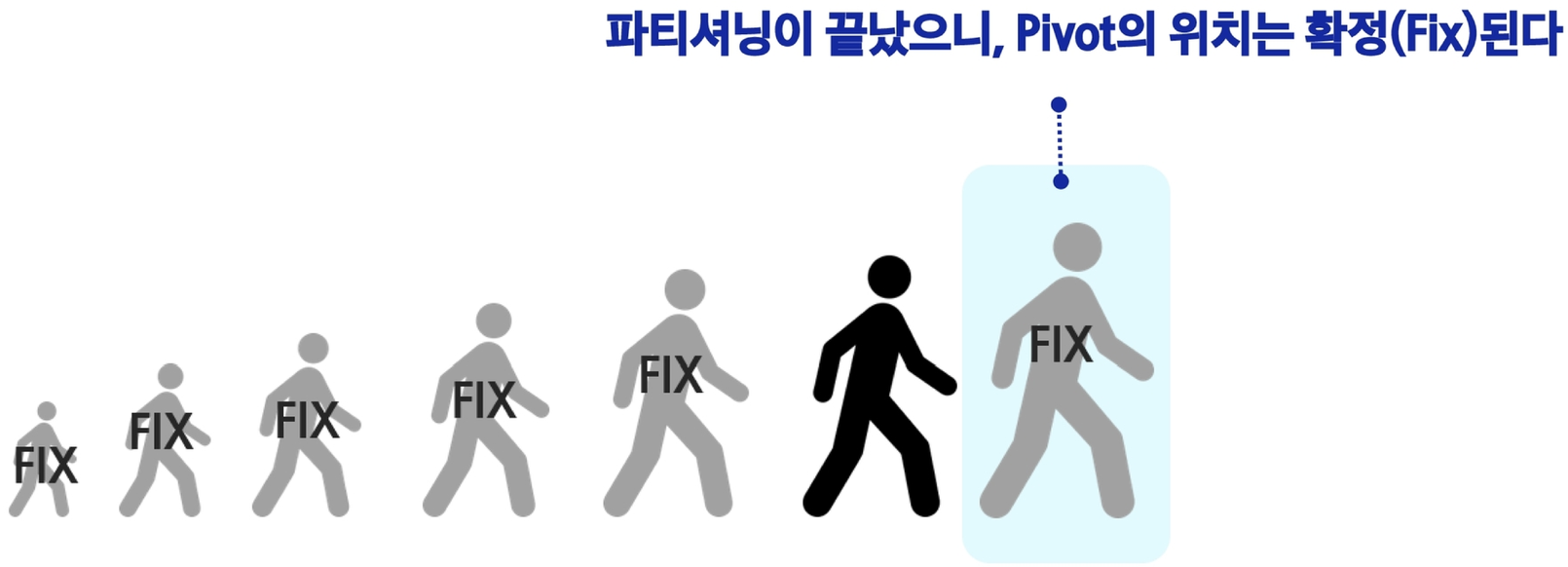
1. 작업 영역 지정
2. Pivot 결정
3. **Pivot을 기준으로 배치 변경**



<그림24_알고리즘 응용_분할정복_파티셔닝 21>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

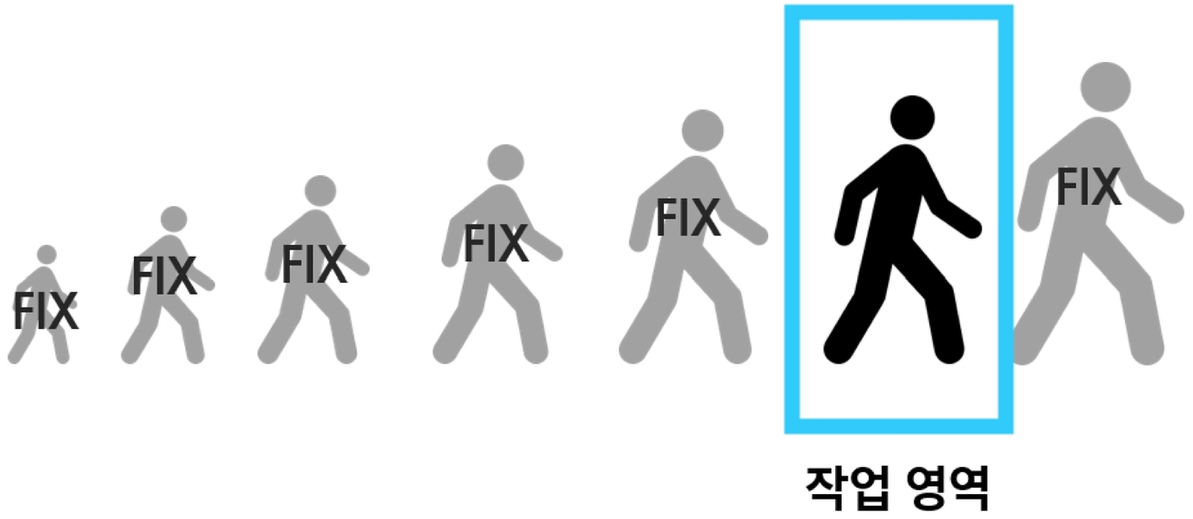


<그림25_알고리즘 응용_분할정복_파티셔닝 22>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

1. **작업 영역 지정**
2. Pivot 결정
3. Pivot을 기준으로 배치 변경



<그림26_알고리즘 응용_분할정복_파티셔닝 23>

Partitioning

- 한 번의 파티셔닝 이후, 왼쪽과 오른쪽 부분 배열에 대해 재귀적으로 파티셔닝을 반복하여 정렬을 진행합니다.

작업 영역에 혼자이므로, 해당 값이 Fix가 되고 바로 파티셔닝이 종료된다.

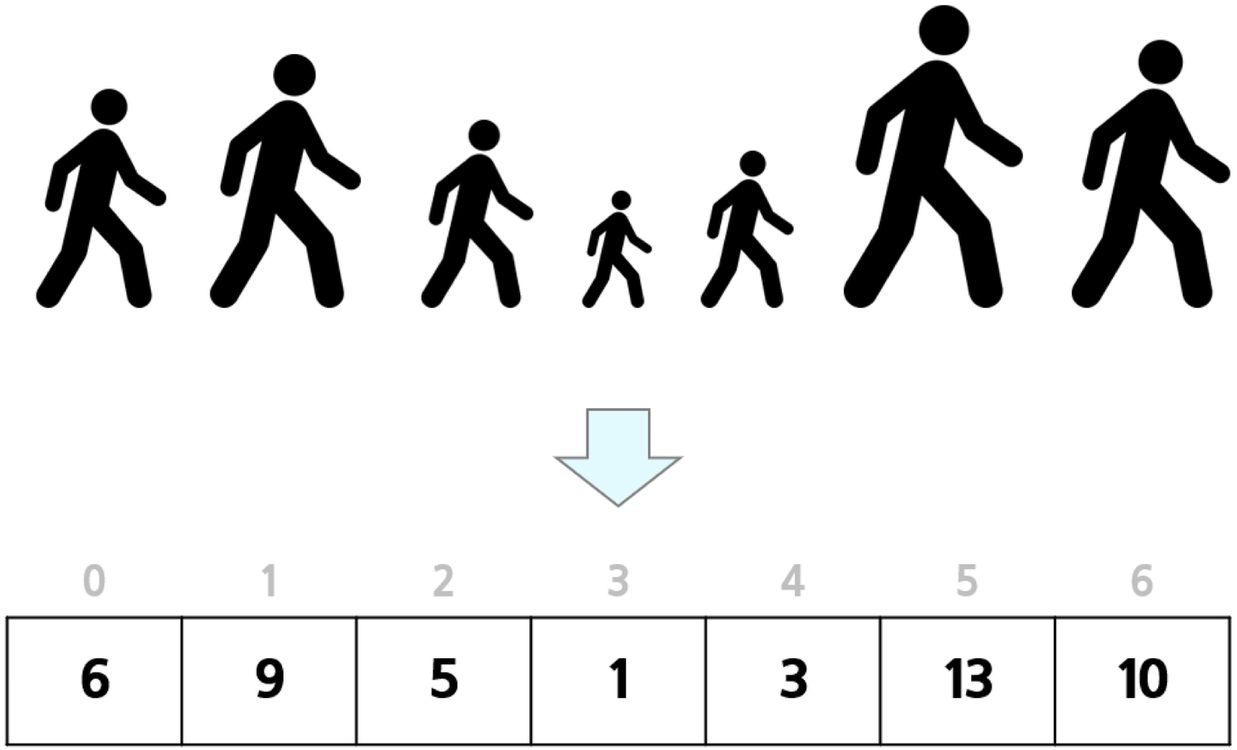
모든 값들의 위치가 Fix 되었다.

이렇게 Partitioning을 반복하여 Quick Sort가 진행된다.

<그림27_알고리즘 응용_분할정복_파티셔닝 24>

Partitioning

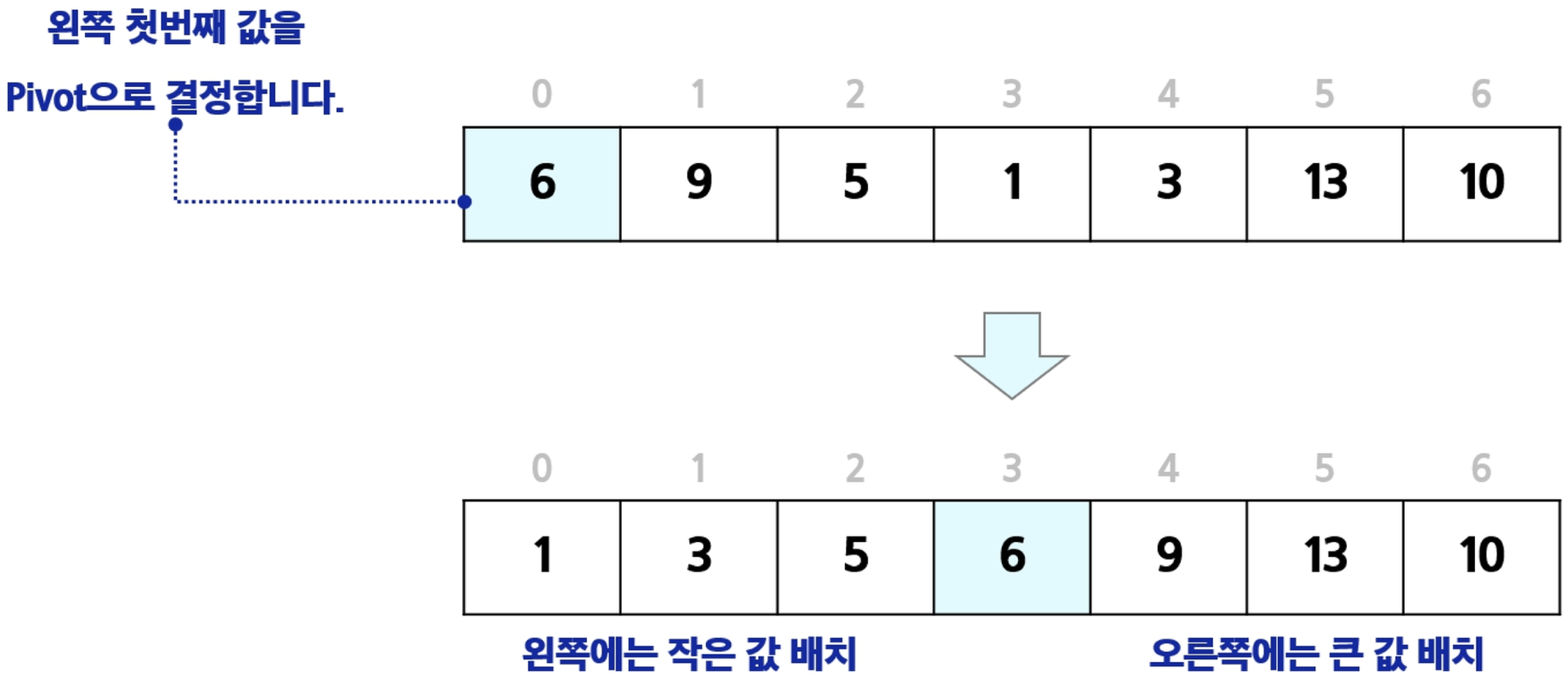
- 리스트에서 파티셔닝 하기
  - 앞서 설명한 내용을 리스트로 표현하면 다음과 같습니다



<그림28_알고리즘 응용_분할정복_리스트에서 파티셔닝 하기 1>

Partitioning

- 리스트에서 파티셔닝 하기
  - 작업 영역을 0 ~ 6으로 정했을 때 예시



<그림29_알고리즘 응용_분할정복_리스트에서 파티셔닝 하기 2>

Partitioning

- 알고리즘

```
quickSort(A[], l, r)
        if l < r
                s ← partition(A[], l, r)
                quickSort(A[], l, s - 1)
                quickSort(A[], s + 1, r)
```

Hoare-Partition

- 알고리즘

- 코드 : 2.quick_sort_hoare.py

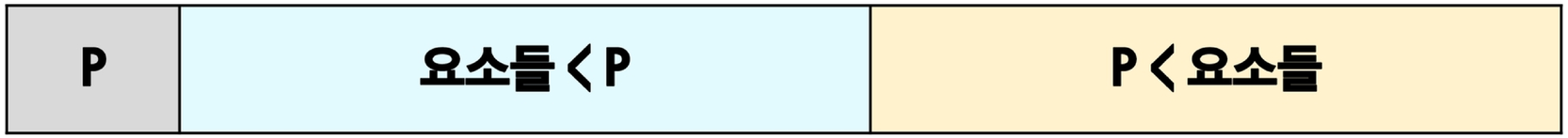
```
partition(A[], l, r)
        p ← A[l]                    // p: 피봇 값
        i ← l,  j ← r
        WHILE i ≤ j
                WHILE i ≤ j and A[i] ≤ p : i++
                WHILE i ≤ j and A[j] ≥ p : j--
                IF i < j : swap(A[i], A[j])

        swap(A[l], A[j])
        RETURN j
```

Hoare-Partition

- 아이디어

※ P(피봇)값들 보다 큰 값은 오른쪽, 작은 값들은 왼쪽 집합에 위치하도록 합니다.



※ 피봇을 두 집합의 가운데에 위치시킵니다.

Hoare-Partition

- 왼쪽 끝/오른쪽 끝/가운데 세 값 중에 중간 값을 선택하는 경우

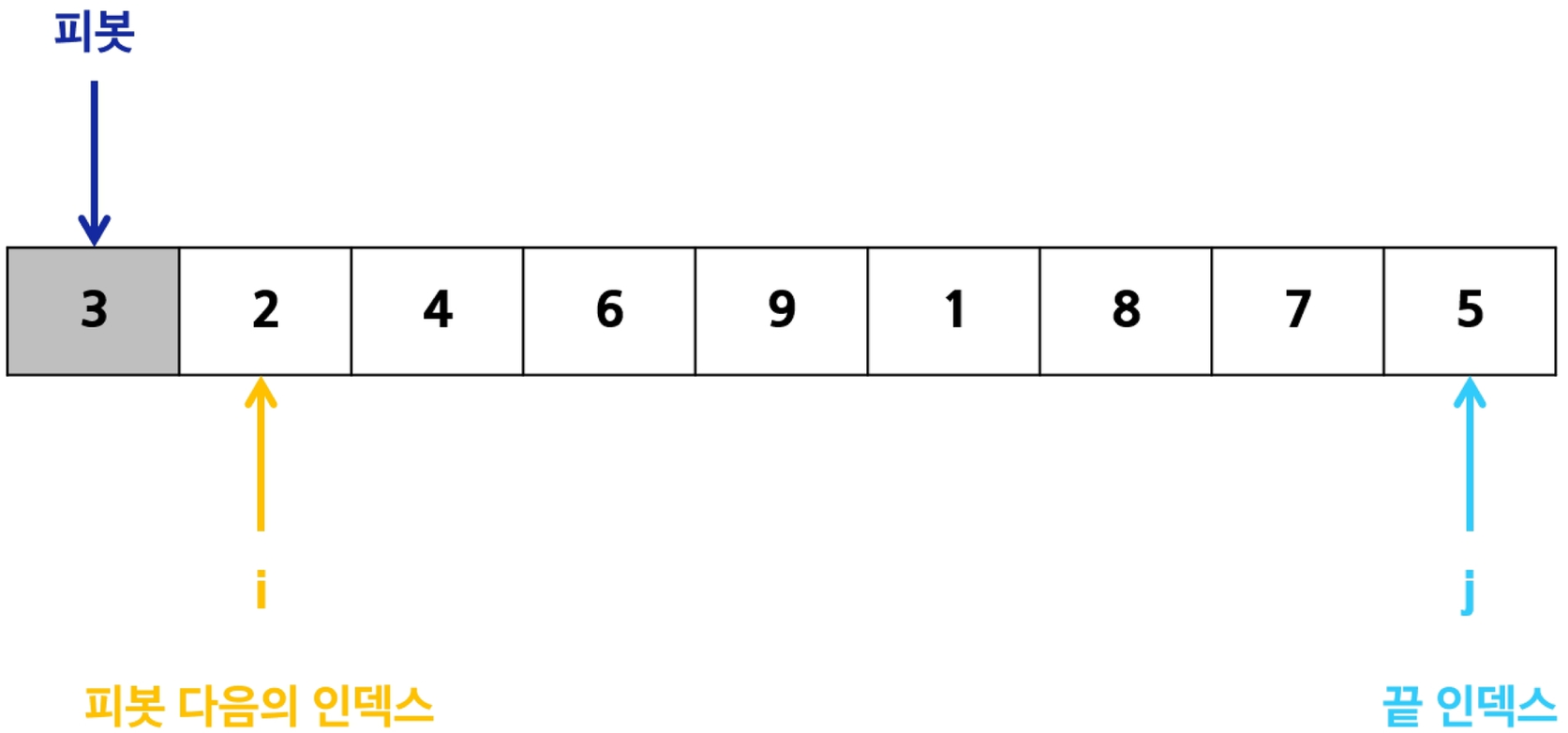
3	2	4	6	9	1	8	7	5

3	2	4	6	9	1	8	7	5

3	2	4	6	9	1	8	7	5

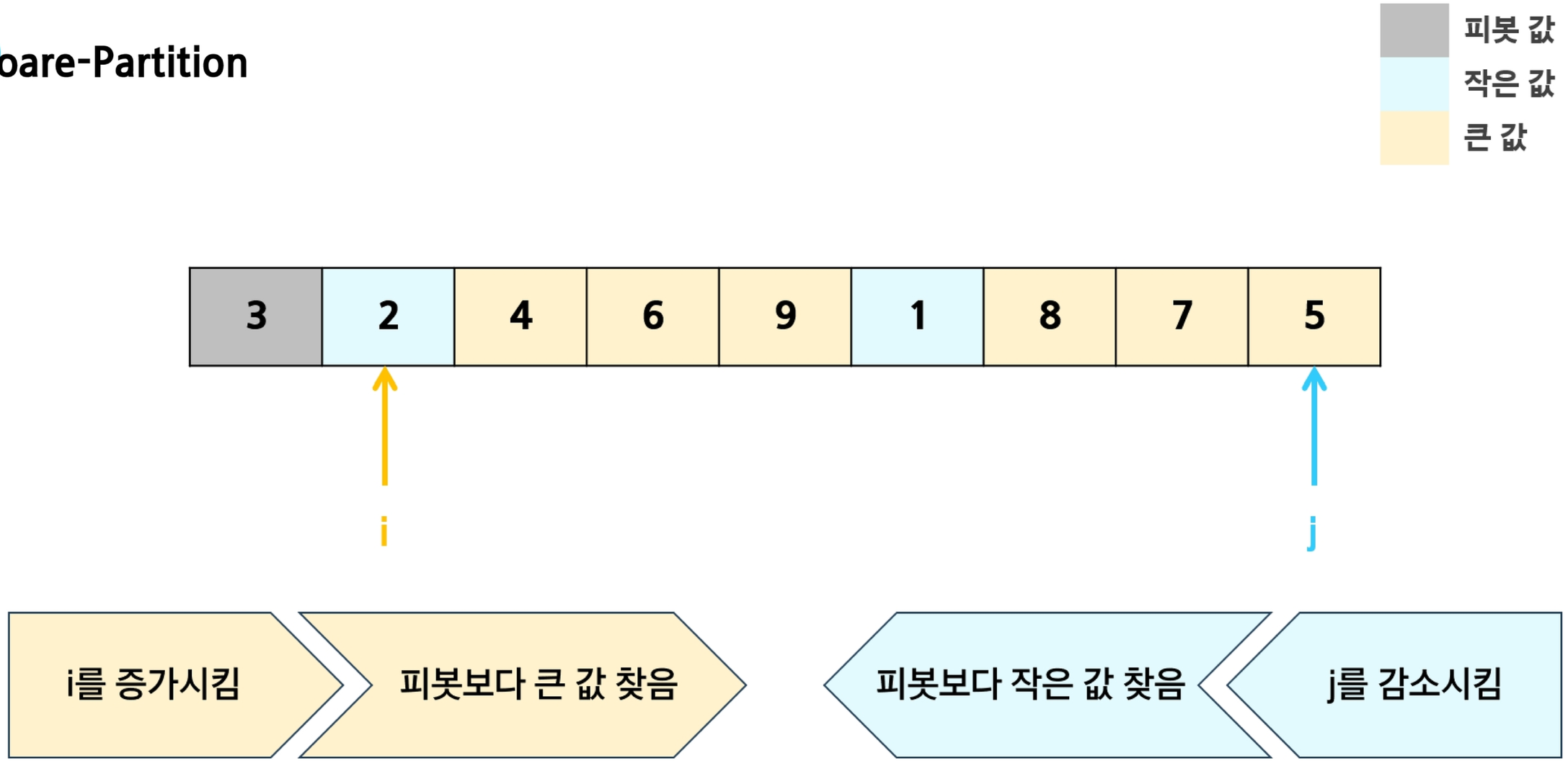
5	2	4	6	9	1	8	7	3

Hoare-Partition



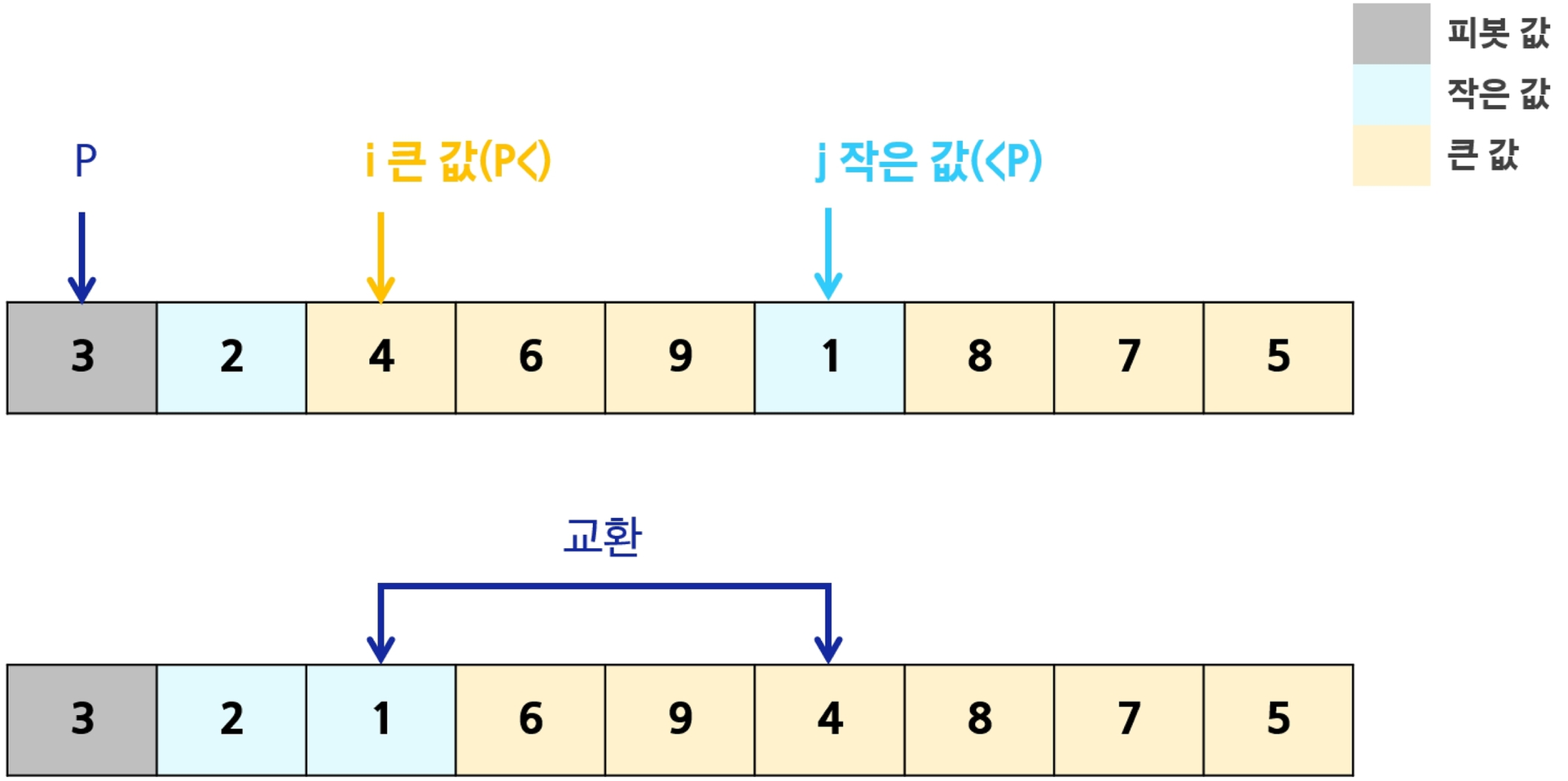
<그림30_알고리즘 응용_분할정복_Hoare-Partition 1>

Hoare-Partition



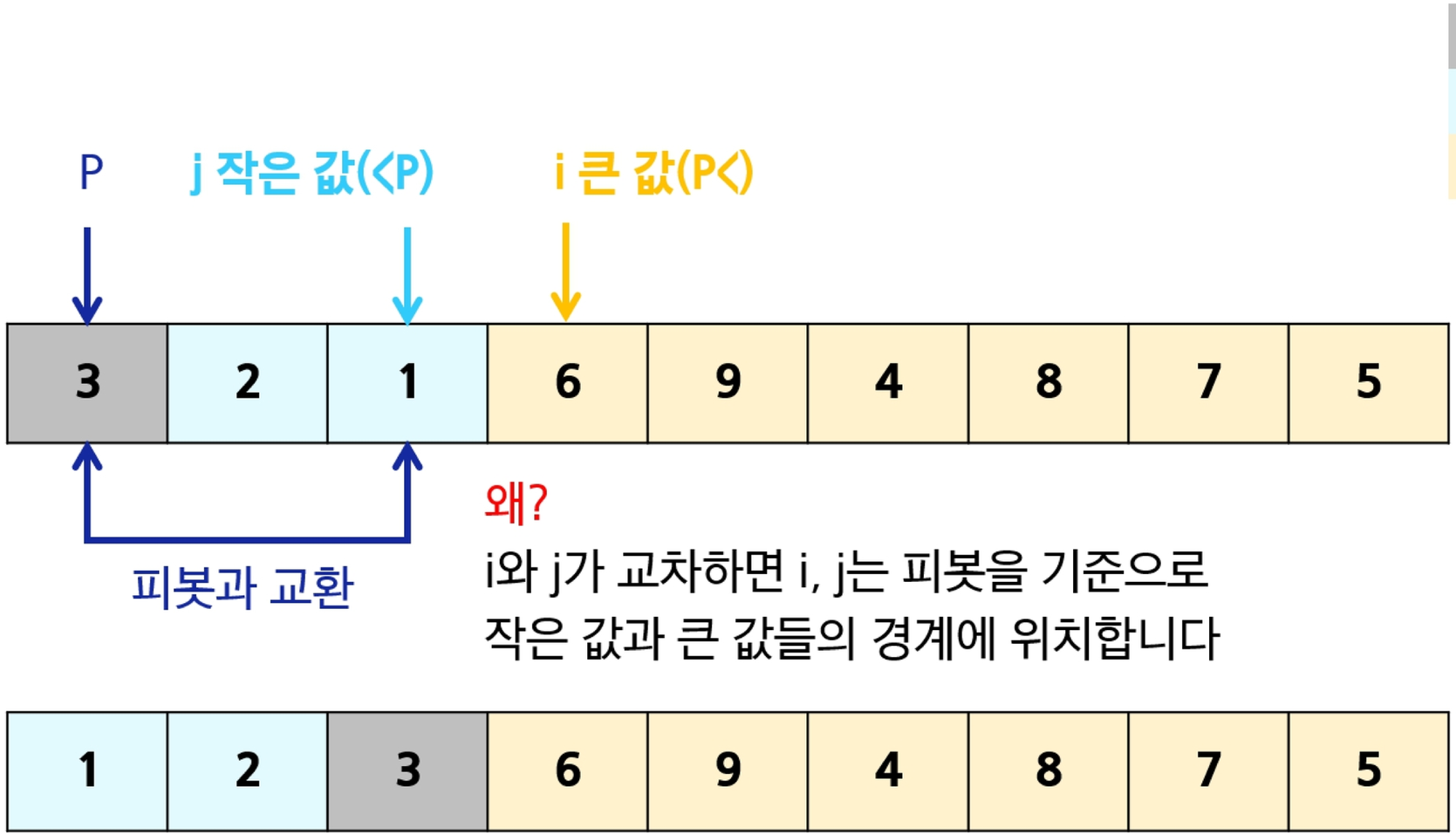
<그림31_알고리즘 응용_분할정복_Hoare-Partition 2>

Hoare-Partition



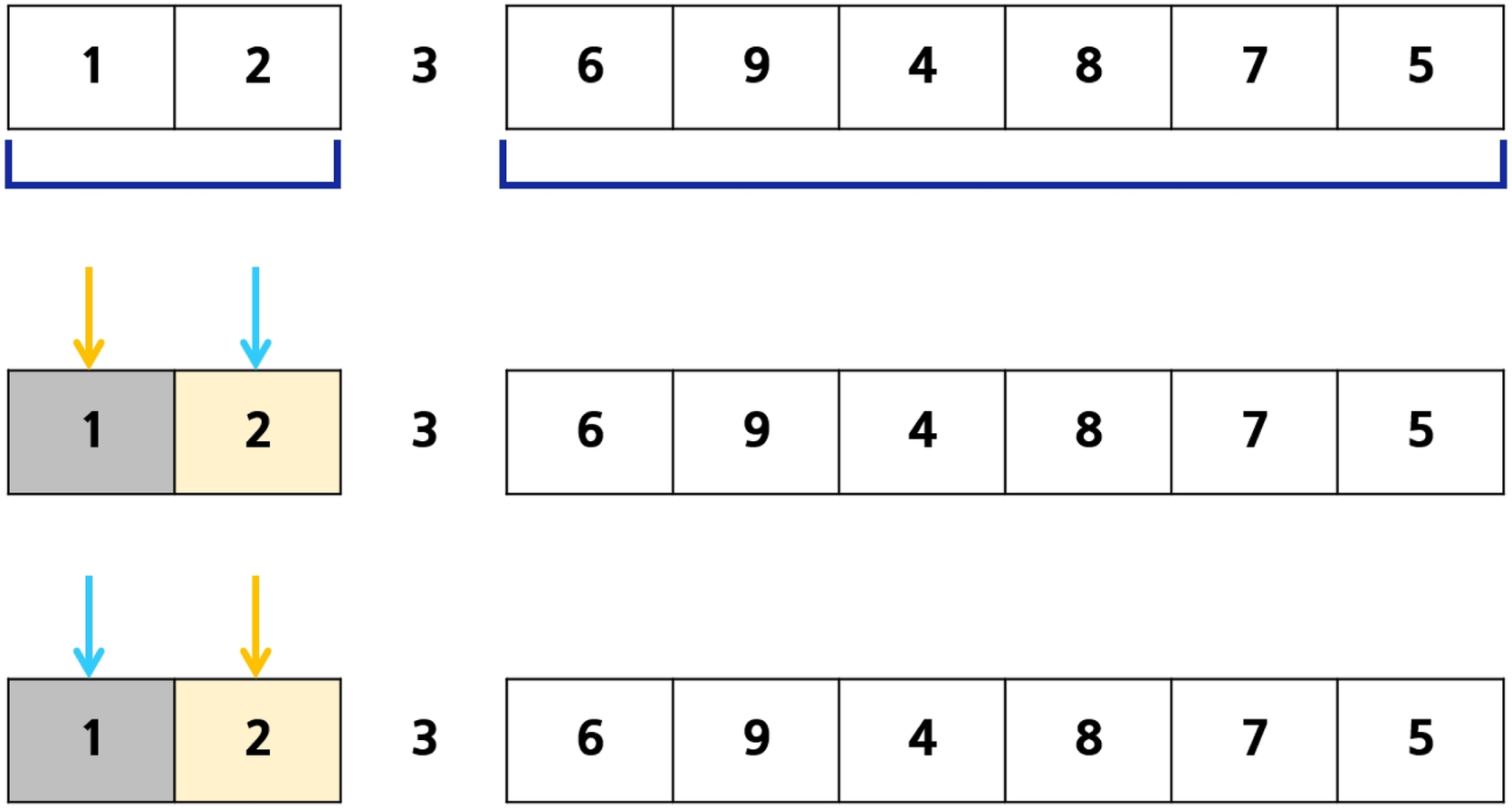
<그림32_알고리즘 응용_분할정복_Hoare-Partition 3>

Hoare-Partition



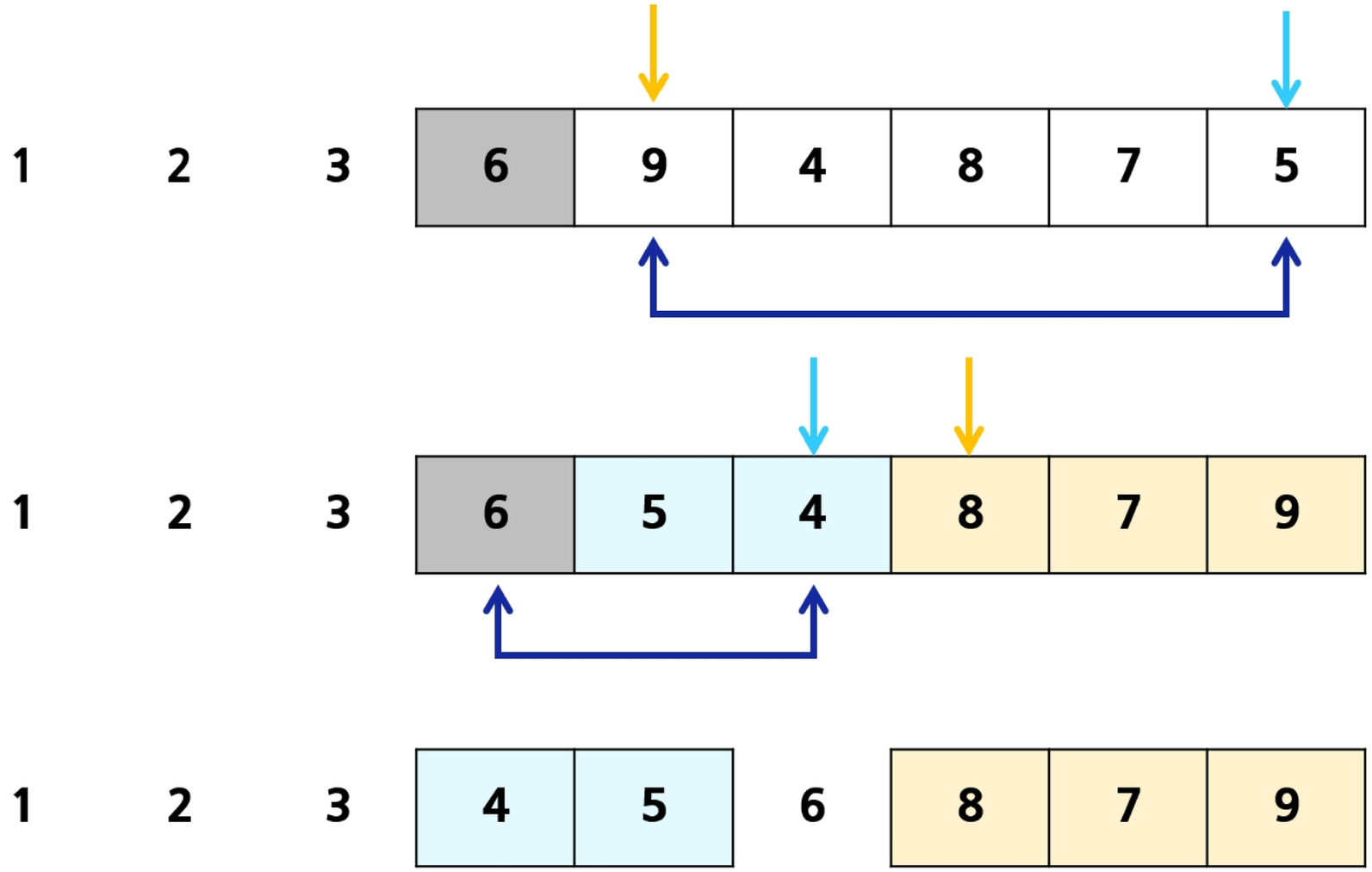
<그림33_알고리즘 응용_분할정복_Hoare-Partition 4>

Hoare-Partition



<그림34_알고리즘 응용_분할정복_Hoare-Partition 5>

Hoare-Partition



<그림35_알고리즘 응용_분할정복_Hoare-Partition 6>

Lomuto Partition

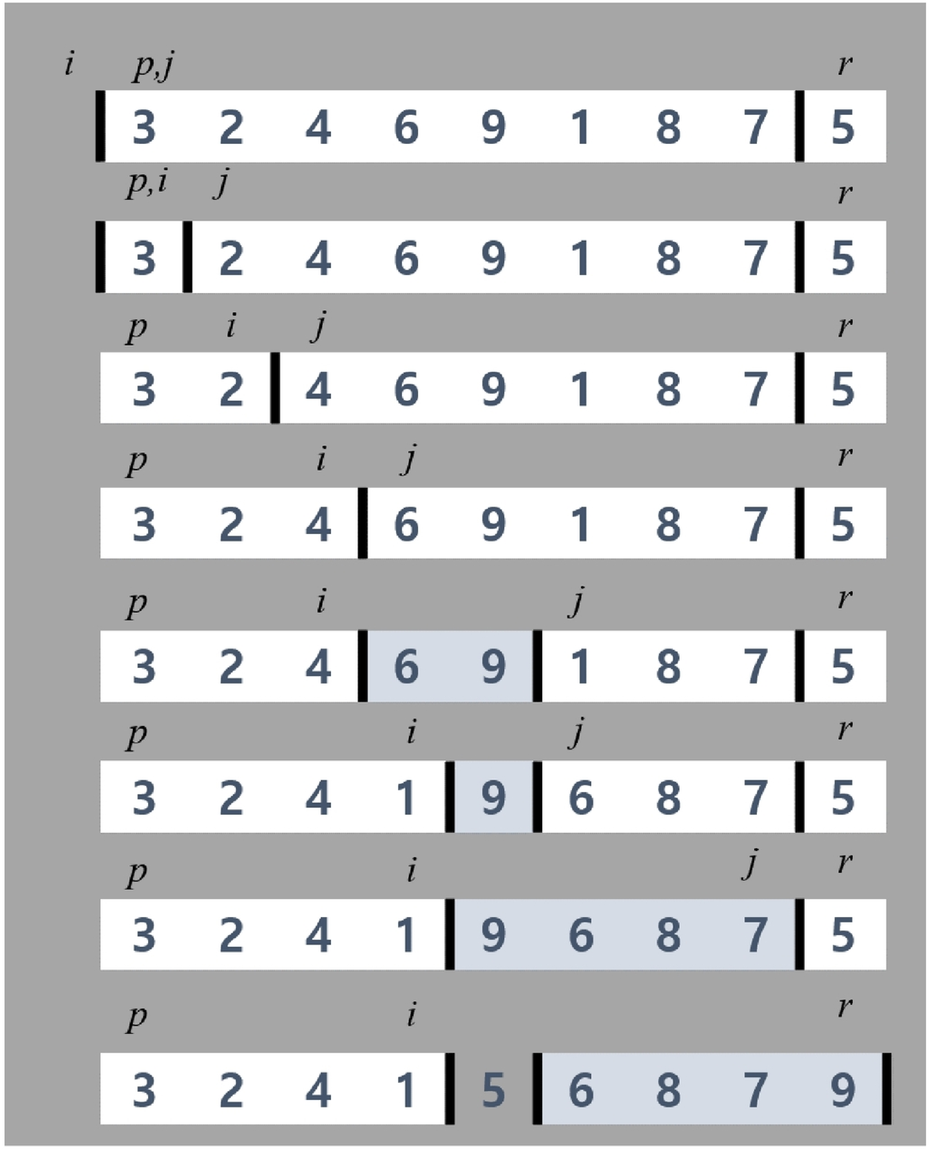
- 알고리즘

- 코드 : 2.quick_sort_lomuto.py

```
partition(A[], p, r)
    x ← A[r]
    i ← p – 1

    FOR j in p → r – 1
        IF A[j] ≤ x
            i++
            swap(A[i], A[j])

    swap(A[i+1], A[r])
    RETURN i + 1
```



<그림36_알고리즘 응용_분할정복_Lomuto Partition>

연습문제

배열의 데이터를 퀵 정렬하는 함수를 작성하고 테스트 해봅시다.

입력 예

11 45 23 81 28 34

11 45 22 81 23 34 99 22 17 8

1 1 1 1 1 0 0 0 0 0

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

이진 검색

병뚜껑 속의 숫자 게임

술래가 병뚜껑 속 숫자를 확인한 후, 다음 사람부터 숫자를 맞히기 시작한다. 술래는 Up 또는 Down을 통해 게임에 참여한 사람들이 병뚜껑 속 숫자에 점점 가까워질 수 있도록 힌트를 제시한다.

- 예시) * 병뚜껑 속 숫자가 3일 경우

첫 번째 사람이 14를 외쳤다면! ▶ 술래는 'DOWN'! ▶ 두 번째 사람이 2를 외쳤다면! ▶ 술래는 'UP'! ▶ 세 번째 사람이 4를 외쳤다면! ▶ 술래는 'DOWN'! ▶ 결국 네 번째 사람이 병뚜껑 속 숫자인 3을 부를 수 밖에 없으므로 벌주 당첨!

최대로 빨리 당첨 되려면 어떻게 하면 될까?

<그림37_알고리즘 응용_분할정복_병뚜껑 속의 숫자 게임>

이진 검색(Binary Search)의 개념

이진 검색
Binary Search

자료의 가운데에 있는 항목의 키 값과 비교하여
다음 검색의 위치를 결정하고 검색을 계속 진행하는 방법

목적 키를 찾을 때까지 이진 검색을 순환적으로 반복 수행함으로써
검색 범위를 반으로 줄여가면서 보다 빠르게 검색을 수행합니다

이진 검색의 과정

① 자료의 중앙에 있는 원소를 고릅니다.

② 중앙 원소의 값과 찾고자 하는 목표 값을 비교합니다.

③ 목표 값이 중앙 원소의 값보다 작으면 자료의 왼쪽 반에 대해서 새로 검색을 수행하고,
크다면 자료의 오른쪽 반에 대해서 새로 검색을 수행합니다.

④ 찾고자 하는 값을 찾을 때까지 ①~③의 과정을 반복합니다.

※ **이진 검색을 하기 위해서는 자료가 정렬된 상태여야 합니다.**

이진 검색의 과정

- 예) 이진 검색으로 7을 찾는 경우

<검색 과정>

① 7 < 9
⇒ 왼쪽 검색

② 7 > 4
⇒ 오른쪽 검색

③ 7 = 7
⇒ 검색 성공!

<그림38_알고리즘 응용_분할정복_이진 검색 과정 1>

이진 검색의 과정

- 예) 이진 검색으로 20을 찾는 경우
- 20 > 9

<검색 과정>

① 20 > 9

⇒ 오른쪽 검색

② 20 > 19

⇒ 오른쪽 검색

③ 20 ≠ 23

⇒ 검색 실패!

<그림39_알고리즘 응용_분할정복_이진 검색 과정 2>

이론

이진 검색 알고리즘 : 반복 구조

2	4	7	9	11	19	23

- 코드 : 3.binary_search.py

```
binary_search(n, S[], key):
    low ← 0
    high ← n – 1

    WHILE low <= high
        mid ← (low + high) / 2

        IF S[mid] == key
            RETURN mid
        ELIF S[mid] > key
            high ← mid – 1
        ELSE
            low ← mid + 1
    RETURN -1
```

이진 검색 알고리즘 : 재귀 구조

```
binary_search(a[], low, high, key)
    IF low > high
        RETURN -1
    ELSE
        mid ← (low + high) / 2
        IF key == a[mid]
               RETURN mid
        ELIF key < a[mid]
               RETURN binary_search(a[], low, mid - 1, key)
        ELSE
               RETURN binary_search(a[], mid + 1, high, key)
```

분할 정복 알고리즘 정리

- 병합 정렬

 - 외부 정렬(External Sort)의 기본이 되는 정렬 알고리즘 입니다.

 - 멀티코어(Multi-Core) CPU 나 다수의 프로세서에서 정렬 알고리즘을 병렬화 하기위해 병합 정렬 알고리즘이 활용됩니다.

- 퀵 정렬

 - 매우 큰 입력 데이터에 대해서 좋은 성능을 보이는 알고리즘 입니다.

- 이진 검색

 - 정렬된 데이터를 기준으로 특정 값이나 범위를 검색하는 데 사용합니다.

 - [이진 검색을 활용한 심화 학습 키워드] Lower Bound, Upper Bound

    - 정렬된 배열에서 특정 값 이상(이하) 가 처음으로 나타나는 위치를 찾는 알고리즘
    - 특정 데이터의 범위 검색 등에서 활용

실습

분할정복

- 5204. 병합 정렬
- 5207. 이진 탐색

퀵 정렬

- 5205. 퀵 정렬

추가 연습

- 5688. 세제곱근을 찾아라
- 14229. 백만 개의 정수 정렬 (user problem)

Test

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 질문에 답하시오. (단답형)

이진 검색이 가능하려면 데이터는 어떤 상태여야 하나요?

2 다음 중 분할정복 알고리즘이 아닌 것은?

a) 병합 정렬
b) 퀵 정렬
c) 선택 정렬
d) 이진 탐색

3 다음 중 퀵 정렬의 평균 시간 복잡도는?

a) O(n)
b) O(n log n)
c) O(n²)
d) O(log n)

4 다음 질문에 답하시오. (단답형)

퀵 정렬에서 피봇의 역할은?

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 병합 정렬은 다음 중 어떤 구조로 구현하는가?

a) 반복문
b) 큐
c) 재귀
d) 우선순위 큐

6 다음 질문에 답하시오. (단답형)

퀵 정렬에서 다음 배열 [7, 3, 5, 2, 9]의 첫 번째 원소를 피봇으로 정했을 때, 왼쪽 그룹에 들어가는 값은?

7 이진 탐색은 어떤 방식으로 탐색 범위를 줄이나요?

a) 앞에서부터 하나씩 확인
b) 무작위 탐색
c) 중간 값을 기준으로 절반씩 제거
d) 끝값 기준으로 탐색

8 다음 질문에 답하시오. (단답형)

이진 탐색을 재귀로 구현할 때 재귀 호출이 끝나는 조건은?

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 병합 정렬은 정렬을 위해 어떤 추가 자료구조를 사용하나요?

a) 큐
b) 트리
c) 임시 배열
d) 힙

10 분할정복 전략을 사용하는 알고리즘의 공통점은?

a) 정렬되어 있어야만 작동한다
b) 입력 크기를 항상 3개로 나눈다
c) 동일한 방식으로 작은 문제를 해결하고 필요에 따라 병합한다
d) 반복문만 사용된다

11 다음 질문에 답하시오. (단답형)

이진 탐색에서 탐색 구간이 low > high가 되었을 때 의미하는 것은?

12 다음 중 병합 정렬의 특징으로 옳지 않은 것은?

a) 안정 정렬이다
b) 추가 메모리를 사용한다
c) 제자리 정렬이다
d) 항상 O(n log n)의 시간 복잡도를 가진다

확인문제

정답 및 해설

1. 정렬된 상태 2. c) 선택 정렬 3. b) O(nlog n) 4. 작은 값과 큰 값을 분할하는 기준 5. c) 재귀 6. 3, 5, 2 7. c) 중간 값을 기준으로 절반씩 제거 8. 중간값이 찾는 값과 같거나 시작 인덱스가 끝 인덱스 보다 큰 경우 9. c) 임시 배열 10. c) 동일한 방식으로 작은 문제를 해결하고 필요에 따라 병합한다 11. 탐색 실패 (찾는 값 없음) 12. c) 제자리 정렬이다

1. 이진 탐색은 중간값을 기준으로 탐색 범위를 절반씩 줄이기 때문에, 데이터가 정렬되어 있어야 합니다.

2. 선택 정렬은 단순 반복을 통해 최소값을 찾아가는 방식이며, 분할정복 전략을 사용하지 않습니다.

3. 퀵 정렬은 평균적으로 분할과 정복을 효율적으로 수행하여 O(n log n)의 시간 복잡도를 가집니다.

4. 퀵 정렬은 피봇을 기준으로 작은 값과 큰 값을 나눠 정렬의 기반을 마련합니다.

5. 병합 정렬은 함수가 자기 자신을 호출하는 재귀 구조로 구현되는 것이 일반적입니다.

6. 피봇이 7일 경우, 7보다 작은 값인 3, 5, 2는 왼쪽 그룹에 속하게 됩니다.

7. 이진 탐색은 항상 중간값과 비교하여 탐색 범위를 반으로 줄여나갑니다.

8. low > high일 때 탐색 실패로 끝나는 경우,중간 값이 찾는 값과 일치할 때 탐색 성공으로 끝나는 경우입니다.

9. 병합 정렬은 정렬하면서 새로운 배열에 임시 저장하며 병합합니다.

10. 분할정복은 문제를 나누고, 동일한 방식으로 해결한 뒤 합치는 전략입니다.

11. 탐색 범위가 역전되면 탐색 대상이 사라졌음을 의미합니다.

12. 병합 정렬은 추가 배열을 사용하므로 제자리(in-place) 정렬이 아닙니다.

활동정리

핵심키워드

개념	설명	예시
분할 정복	문제를 작은 하위 문제로 나누고(분할) 각각을 해결(정복)한 뒤, 그 결과를 필요에 따라 결합(통합)하여 원래 문제를 해결하는 알고리즘 기법	병합 정렬, 퀵 정렬, 이진검색 등
병합 정렬	여러 개의 정렬된 자료의 집합을 병합하여 한 개의 정렬된 집합으로 만드는 방식	[8,3,5,2]→[8,3][5,2]→[8][3][5][2] → [3,8][2,5] → [2,3,5,8]
퀵 정렬	기준값을 중심으로 주어진 배열을 두 개로 분할하고, 각각을 정렬하여 전체 배열을 정렬하는 방식	[6, 3, 7, 1] → [3,1] 6 [7] → 피봇 좌우 구간에 대해 파티셔닝
이진 검색	정렬된 자료의 가운데에 있는 항목의 키 값과 비교하여 다음 검색의 위치를 결정하고 검색을 계속 진행하는 방법	[1, 3, 5, 7, 9]에서 7 찾기: 중간(5) < 7 →오른쪽 구간 탐색

<표2_알고리즘 응용_분할정복_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **분할 정복**
  - 문제를 분할해서 해결하는 기법
  - 대표적으로 병합 정렬, 퀵 정렬, 이진 검색 알고리즘이 있음
  - 대표적인 큐의 종류로 선형 큐, 원형 큐, 연결 큐, 우선순위 큐가 있음

- **병합 정렬**
  - 여러 개의 정렬된 자료의 집합을 병합하여 한 개의 정렬된 집합으로 만드는 방식
  - 시간 복잡도 : O(n log n)

활동정리

오늘 공부한 내용 요약 및 정리

- **퀵 정렬**
  - 기준값(Pivot)을 중심으로 주어진 배열을 두 개로 분할하고, 각각을 정렬하여 전체 배열을 정렬하는 방식
  - 각 부분 정렬이 끝난 후, 병합정렬은 “병합”이란 후처리 작업이 필요하나, 퀵 정렬은 필요로 하지 않음
  - 평균 시간 복잡도 O(n log n), 최악의 경우 $O(n^2)$
  - Hoare-Partition 알고리즘, Lomuto-Partition 알고리즘이 있음

- **이진 검색**
  - 자료의 가운데에 있는 항목의 키 값과 비교하여 다음 검색의 위치를 결정하고 검색을 계속 진행하는 방법
  - 이진 검색을 하기 위해서는 자료가 정렬된 상태여야 함

활동정리

퍼즐을 맞추는 가장 효율적인 순서는?

퍼즐 100조각을 한 번에 맞추기 어렵습니다. 그래서 아래와 같은 단계를 고려해보았습니다. 아래 전략은 어떤 순서로 이루어졌을까요?

: 색깔이나 테두리 등으로 조각을 분류함

: 각 분류된 조각 그룹을 따로따로 맞춤

: 맞춘 조각 그룹을 하나로 이어 전체 퍼즐 완성

활동정리

퍼즐을 맞추는 가장 효율적인 순서는?

퍼즐 100조각을 한 번에 맞추기 어렵습니다. 그래서 아래와 같은 단계를 고려해보았습니다. 아래 전략은 어떤 순서로 이루어졌을까요?

분할 : 색깔이나 테두리 등으로 조각을 분류함

정복 : 각 분류된 조각 그룹을 따로따로 맞춤

결합 : 맞춘 조각 그룹을 하나로 이어 전체 퍼즐 완성

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다